

Efficient Algorithms for the Uniform Tokenization Problem

ANGELA W. LI, Rice University, USA
KONSTANTINOS MAMOURAS, Rice University, USA

Tokenization (also known as scanning or lexing) is a computational task that has applications in the lexical analysis of programs during compilation and in data extraction and analysis for unstructured or semi-structured data (e.g., data represented using the JSON and CSV data formats). We propose two algorithms for the tokenization problem that have linear time complexity (in the length of the input text) without using large amounts of memory. We also show that an optimized version of one of these algorithms performs well compared to prior approaches on practical tokenization workloads.

CCS Concepts: • **Theory of computation** → **Formal languages and automata theory**; **Regular languages**; • **Software and its engineering** → *Semantics*.

Additional Key Words and Phrases: lexical analysis, scanning, tokenization, maximal munch semantics, NFA, nondeterministic finite automata, backtracking

ACM Reference Format:
Angela W. Li and Konstantinos Mamouras. 2025. Efficient Algorithms for the Uniform Tokenization Problem. *Proc. ACM Program. Lang.* 9, OOPSLA1, Article 133 (April 2025), 27 pages. https://doi.org/10.1145/3720498

1 Introduction

Tokenization, also known as scanning or lexing, is the computational task of splitting a string into a sequence of substrings called tokens. The rules that specify how the string should be split into tokens are typically given as a list of regular expressions. For example, the tokenization rules

```
[a-zA-Z]+    [0-9]+    [,!;\.]    [ \t\n]+
```

describe token classes for words, numbers, punctuation symbols, and whitespace respectively. A tokenizer (scanner or lexer), is a program that performs tokenization on a given input string. Such a program can be handcrafted or automatically generated by tools commonly referred to as lexer generators. In particular, a lexer generator takes as input a user-defined tokenization grammar and outputs a lexer that performs tokenization according to the user-supplied grammar. Some popular lexer generators used in practice include flex [Vern Paxson 1987] and Ocamllex [Smith 2007].

Tokenization has applications in several domains, including lexical analysis in compilers, natural language processing, and the processing of text-based datasets (e.g., JSON, CSV, and XML) in data science. Lexical analysis is typically not the computational bottleneck in the compilation of programs. The measurements reported in [Barenghi et al. 2015] suggest that tokenization constitutes roughly 25-41% of the computational cost in large data querying. This suggests that efficient tokenization algorithms can speed up the processing of large text-based data.

While many common data formats are standardized, we argue that creating a specialized tokenizer for each data format does not offer sufficient flexibility. Users sometimes would like to adapt existing data-exchange formats for their own use cases, or they may even want to create their own

Authors' Contact Information: Angela W. Li, Rice University, Houston, USA, awl@rice.edu; Konstantinos Mamouras, Rice University, Houston, USA, mamouras@rice.edu.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2025 Copyright held by the owner/author(s).
ACM 2475-1421/2025/4-ART133
https://doi.org/10.1145/3720498

data-representation language and corresponding grammar. For example, standard data-exchange languages, such as JSON [JSON 2001] and XML [XML 1996], have a commonly agreed-upon syntax and grammars that facilitate interoperability. We have found, however, anecdotal evidence that developers would like JSON to allow comments[1] and trailing commas[2]. These criticisms have mostly gone unanswered by the creators of JSON, and developers have come up with ad-hoc ways of adapting JSON to meet their needs. One common workaround for the lack of comments in JSON is defining a key-value pair of the form `"_comment" : "comment body"`[3]. Alternatively, one may use JSON5, a proposed superset of JSON that supports additional features such as single-quote strings, trailing commas, comments, and more flexible representations [JSON5 2012]. However, not every user is happy with JSON5, either. For instance, JSON5 still does not support date/time literals as a separate syntactic category[4]. For the sake of readability, some users would also have preferred that JSON5 should treat newlines and white space as delimiters[5]. Clearly, there are no one-size-fits-all solutions to such dissatisfactions about data exchange formats. This calls for a flexible approach for accommodating each user's individual needs in data representation. For this reason, we argue that users should be able to define custom grammars, which may be completely original or an adaptation of an existing grammar like JSON, to suit their own needs.

A common approach when processing semi-structured data is to perform full parsing of a dataset to generate an in-memory tree representation (i.e., an abstract syntax tree or AST) of the data, which is then used for further analysis. For a large dataset, full parsing can be expensive, both in terms of time and memory needed to create the AST. There are, however, several use cases, where tokenization can serve as an efficient way to extract information from a big dataset. If one would like to extract only a small amount of context-independent information from a dataset, one could use a standalone tokenizer (with a carefully crafted custom grammar), which is computationally cheaper than performing full parsing. As an example, to extract from a JSON document all strings that represent U.S. phone numbers, one can tokenize the file using the regular expression `"[0-9]{3}-[0-9]{3}-[0-9]{4}"` (in addition to the other rules of the JSON tokenization grammar). The token stream produced by the tokenizer can be filtered to produce the stream of string tokens that represent phone numbers. This approach avoids context-free parsing that would generate unnecessary additional data structures for representing objects, arrays, and other irrelevant values. The takeaway is that, depending on the specific task, standalone tokenization may be a better way to extract information compared to fully parsing and querying the AST.

In this paper, we study the ***uniform tokenization problem***, which takes as input a nonempty list $\bar{r} = [r_0, r_1, \ldots, r_{\kappa-1}]$ of regular expressions ("tokenization grammar") and a string $w \in \Sigma^*$ (where Σ is the input alphabet) and returns a partition of w as a list of tokens. The qualifier "uniform" refers to the fact that the tokenization rules $\bar{r}$ are part of the input, which means that they are *not* considered to be fixed for the computational problem. This is different from considering a *family* of tokenization problems, one for each tokenization grammar, where the grammar is considered to be fixed (hence its size is a constant) and only the string is considered to be input. This "uniform" viewpoint has an important implication for complexity accounting: our time and space complexity bounds need to include as an additional parameter the size m of the tokenization grammar.

The standard tool flex [Vern Paxson 1987] is a lexer generator that takes as input a tokenization grammar and produces C code that tokenizes an input string according to the grammar. Even though the way flex is used corresponds to the non-uniform version of the tokenization problem,

[1] https://www.reddit.com/r/ProgrammerHumor/comments/ubj77x/json_please/
[2] https://www.reddit.com/r/ProgrammerHumor/comments/tg9oo7/json_w_trailing_commas_would_be_nice/
[3] https://stackoverflow.com/questions/244777/can-comments-be-used-in-json
[4] https://www.reddit.com/r/programming/comments/hb6b0b/the_json5_data_interchange_format_json5_is_a/
[5] https://github.com/json5/json5/issues/157

we can easily take the algorithmic ideas behind it and formulate the corresponding algorithm for the uniform tokenization problem. The main idea is to construct a DFA for the tokenization grammar, through a standard translation from regular expressions to an NFA and then to a DFA using the subset construction (also known as powerset construction). For typical use cases of lexers, the constructed DFAs are of modest size. However, in the worst case, the DFA size is (unavoidably) exponential in m, which can be impractically large. Flex implements the so-called ***maximal munch*** semantics, which means that we repeatedly find the longest token possible. This means that when flex first identifies a token by scanning the text in a left-to-right fashion, it cannot declare it to be the "maximal munch" token. It has to continue reading the input for the possibility that a longer token may be found. Since the search for a longer token can fail, flex ***backtracks*** to the end position of the longest token found and proceeds to the next round from this point. This backtracking means that flex may end up reading the same input more than once. In fact, in the worst case, flex has a time complexity that is *quadratic* in the length of the input text. This shortcoming of flex's algorithm is a known issue. Reps [1998] has proposed a solution for this quadratic complexity problem by using a *memoization table* that enables the backtracking algorithm to avoid trying out paths that will not lead to a token more than once. The algorithm proposed in [Reps 1998] is based on DFAs and has time complexity $O(Mn)$ (assuming that the DFA fits in memory) and space complexity $O(Mn)$, where M is the size of the DFA (number of states) and n is the length of the input string. This means that [Reps 1998] improves upon the time complexity of flex, but it does so with a potentially large cost in memory use.

In this work, we propose new algorithms for the uniform tokenization problem that solve both problems found in existing implementations: (i) the worst-case quadratic complexity (in the input string length) due to backtracking, and (ii) the worst-case exponential complexity (in the grammar size) due to the use of DFAs. Our algorithms use NFAs and they are based on the idea that, for every position i in the input string $w = w_0 w_1 \dots w_{n-2} w_{n-1}$, we can pre-compute a small amount of information about the suffix $w_i w_{i+1} \dots w_{n-1}$ that completely eliminates the need for backtracking. The pre-computation is performed with an initial right-to-left pass over the input string.

Main Contributions. We make the following contributions in this paper:

(1) We propose a tokenization algorithm that pre-computes information about the longest token that can be found from every possible start position in the input string. The tokenization is then performed by "token skipping" using this token length information. This algorithm has time complexity $O(mn)$ and space complexity $O(m + n)$. To the best of our knowledge, the asymptotic complexity of this algorithm is strictly better than prior known algorithms for the uniform tokenization problem.
(2) We propose one more tokenization algorithm that pre-computes, for every position i in the input string $w_0 \dots w_{n-1}$ and every state q of the ε-NFA for the tokenization grammar, whether q can lead to a final state by consuming the nonempty substring $w_i \dots w_j$ for some $j \geq i$. This information (which we think of as given by an oracle) enables backtracking-free tokenization because we can check whether an identified token can be extended to a longer token or not. We refer to this algorithm as the "token extension oracle" algorithm. It has time complexity $O(mn)$ and space complexity $O(mn)$.
(3) We incorporate a performance optimization to the previous algorithm, which reverts to the use of a DFA instead of an NFA when the size of the DFA is not too large. In this case, the algorithm performs tokenization in time $O(n)$ using work space $O(n)$.
(4) We have implemented our proposed algorithms and compared them experimentally against the algorithms of flex and [Reps 1998]. The experimental results show that our algorithms can efficiently handle families of inputs on which flex and Reps' algorithm experience slowdowns.

On realistic benchmarks, our optimized "extension oracle" algorithm is the most time-efficient and is 2 to 3 times faster than flex, the second-best performing algorithm.

A limitation of the algorithms proposed in this paper is that they are not streaming, since they require a reverse (right-to-left) pass over the input text.

Paper Outline. Section 2 defines the uniform tokenization problem. Section 3 describes tokenization NFAs, a variant of ε-NFAs that can be used for tokenization. Section 4 defines the "longest match" problem and presents an efficient algorithm for it. In Section 5, we describe our efficient algorithms for tokenization. Section 6 presents an experimental evaluation of our proposed algorithms. In Section 7 we discuss related work. We conclude in Section 8 with a summary of our main contributions.

2 The Uniform Tokenization Problem

Let Σ be a finite alphabet of symbols (letters, characters). A predicate $\sigma \subseteq \Sigma$ is called a *character class*. The set $\mathrm{Reg}(\Sigma)$ of *regular expressions* (regexes) is defined by the following grammar:

$$
\begin{array}{rll}
r, r_1, r_2 ::= & \varepsilon \mid & \text{[empty string]} \\
& \sigma \mid & \text{[character class / predicate on } \Sigma\text{]} \\
& r_1 \mid r_2 \mid & \text{[nondeterministic choice / union]} \\
& r_1 \cdot r_2 \mid & \text{[concatenation]} \\
& r^* \,. & \text{[Kleene star / iteration]}
\end{array}
$$

Concatenation is also written as $r_1 r_2$ to reduce notational clutter. The notation r^+ ("repetition of r at least once") is an abbreviation for rr^*. The notation $r?$ is an abbreviation for $r \mid \varepsilon$. For a regular expression r, the notation r^n is an abbreviation for the concatenation $r \cdot r \cdots r$ (n times). The notation $r\{n\} = r^n$ is also commonly used to describe the repetition of r exactly n times. More generally, we write $r\{m, n\} = r^m (r?)^{n-m}$ to denote the repetition of r from m to n times.

Every regular expression r denotes a language $\mathcal{L}(r) \subseteq \Sigma^*$, defined in the standard way. When $u \in \mathcal{L}(r)$, we say that the string u *matches* the regex r. We also say that the regex r *accepts* the string u when $u \in \mathcal{L}(r)$.

For strings u and v, we write $u \cdot v$ or uv for their concatenation. We write $|w|$ to denote the length of a string w. The empty string (i.e., the string of length 0) is denoted by ε. For a string $w \in \Sigma^*$, we will call a pair $[i, j]$ with $0 \le i \le j \le |w|$ a *location* in w. A *position* in $|w|$ is an index in the range $0, 1, \ldots, |w|$. We write $w[i..j]$ for the substring of w at location $[i, j]$. E.g., for the string $w = abbcabab$ (length $|w| = 8$), we have that $w[0..3] = abb$, $w[1..5] = bbca$, and $w[4..7] = aba$, and $w[5..8] = bab$. We also use the abbreviations $w[..i] = w[0..i]$ and $w[i..] = w[i..|w|]$.

We can also think of a string $w \in \Sigma^*$ as a function from $\mathrm{dom}(w) = \{0, 1, \ldots, n-1\}$ to Σ, where $n = |w|$ and $w(i)$ is the letter at position i. This means that $w = w(0)w(1)\ldots w(n-1)$.

For strings $u, v \in \Sigma^*$, we say that u is a *prefix* of v if $uw = v$ for some $w \in \Sigma^*$. When u is a prefix of v, we write $u^{-1}v$ for the unique string w such that $uw = v$. So, we have that $u \cdot u^{-1}v = v$.

Let A be a set. We define $\mathrm{Option}(A) = \{\mathrm{Some}(a) \mid a \in A\} \cup \{\mathrm{None}\}$. Moreover, $\mathrm{List}(A)$ is the set of lists (i.e., finite sequences) with elements from A. The empty list is written as $[\,]$ and $\cdot$ denotes list concatenation.

Definition 1 (Uniform Tokenization Problem). Let $\bar{r} = [r_0, \ldots, r_{\kappa-1}]$ be a nonempty sequence of regular expressions, which we call a *(tokenization) grammar*. We refer to a regular expression r_i of a grammar as a *rule*. We define the function $\mathrm{token}(\bar{r}) : \Sigma^* \to \mathrm{Option}(\Sigma^* \times \mathbb{T})$ as follows: $\mathrm{token}(\bar{r})(u) = \mathrm{Some}(v, \beta)$, where v is the longest nonempty prefix of u that matches some token

regex and $\beta \in \mathbb{T} = \{0, \ldots, \kappa - 1\}$ is the least index such that $v \in \mathcal{L}(r_\beta)$. We also define the function $\mathsf{tokens}(\bar{r}) : \Sigma^* \to \mathsf{List}(\Sigma^* \times \mathbb{T})$ as follows:

$$\mathsf{tokens}(\bar{r})(u) = [\,], \text{ if } \mathsf{token}(\bar{r})(u) = \mathrm{None}$$
$$\mathsf{tokens}(\bar{r})(u) = [(v, j)] \cdot \mathsf{tokens}(\bar{r})(v^{-1}u), \text{ if } \mathsf{token}(\bar{r})(u) = \mathrm{Some}(v, j)$$

Notice that we are slightly abusing notation and writing $\mathrm{Some}(v, j)$ instead of $\mathrm{Some}((v, j))$ for the sake of readability.

The *uniform tokenization problem* is the following: Given a tokenization grammar $\bar{r}$ and a string $w \in \Sigma^*$ as input, compute the list of tokens $\mathsf{tokens}(\bar{r})(w)$.

Example 2 (Tokenization). Consider the tokenization grammar $\bar{r} = [a, ba^*, c[ab]^*]$ and the input text $w = abaabacabaa$. First, we see that $\mathsf{token}(\bar{r})(w) = \mathrm{Some}(a, 0)$ because the string a matches the regular expression a, and no other prefix of w matches any of the three rules of the grammar. Then, we notice that $\mathsf{token}(\bar{r})(w[1..]) = \mathrm{Some}(baa, 1)$ and $\mathsf{token}(\bar{r})(w[4..]) = \mathrm{Some}(ba, 1)$. Finally, $\mathsf{token}(\bar{r})(w[6..]) = \mathrm{Some}(cabaa, 2)$ because the entire remaining part of the string matches the regular expression $c[ab]^*$ (and this is the longest match possible). So, we conclude that

$$\mathsf{tokens}(\bar{r})(w) = [(a, 0), (baa, 1), (ba, 1), (cabaa, 2)]\,.$$

Notice that the entire input string is tokenized.

Example 3 (CSV Tokenization). RFC4180 [Shafranovich 2005] specifies the tokenization grammar of comma-separated values (CSV) to consist of the following four rules:

`[^,\"\n]*`	[Unquoted Field]	`,`	[Field Delimiter]
`\"([^\"]|\"\")*\"`	[Quoted Field]	`\n`	[Record Delimiter]

When given the CSV document `alpha,"beta"\n"gamma",delta` (which consists of two records, each with two fields) as input, the tokenizer should recognize that `alpha` and `delta` are unquoted fields, `"beta"` and `"gamma"` are quoted fields, the commas are field delimiters, and the newline character `\n` is a record delimiter.

Reversal for Strings and Regular Expressions. The *reverse* of a string $w = a_0 a_1 \ldots a_{n-2} a_{n-1} \in \Sigma^*$ is the string $\mathsf{rev}(w) = a_{n-1} a_{n-2} \ldots a_1 a_0$. Observe that $\mathsf{rev}(u \cdot v) = \mathsf{rev}(v) \cdot \mathsf{rev}(u)$ for all $u, v \in \Sigma^*$. We lift the reversal operation to sets of strings as follows: $\mathsf{rev}(L) = \{\mathsf{rev}(w) \mid w \in L\}$ for every language $L \subseteq \Sigma^*$. Reversal on regular expressions is defined by induction:

$$\mathsf{rev}(\varepsilon) = \varepsilon \qquad \mathsf{rev}(r_1 \mid r_2) = \mathsf{rev}(r_1) \mid \mathsf{rev}(r_2) \qquad \mathsf{rev}(r^*) = \mathsf{rev}(r)$$
$$\mathsf{rev}(\sigma) = \sigma \qquad \mathsf{rev}(r_1 \cdot r_2) = \mathsf{rev}(r_2) \cdot \mathsf{rev}(r_1)$$

For every string $w \in \Sigma^*$, it holds that $w \in \mathcal{L}(r)$ iff $\mathsf{rev}(w) \in \mathcal{L}(\mathsf{rev}(r))$.

Extended Regular Expressions. In this work, we consider the standard syntax of regular expressions with nondeterministic choice, concatenation, and Kleene iteration. The tokenization problem could potentially become more difficult if a richer regex syntax with more constructs is used, for example, by including bounded repetition of the form $r\{m, n\}$ with m, n encoded with decimal or binary notation. Recent work [Kong et al. 2022; Le Glaunec et al. 2023; Wen et al. 2024] investigates the challenges of bounded repetition in the context of regex matching. Lookaround assertions (see, e.g., [Miyazaki and Minamide 2019] and [Chattopadhyay et al. 2025]) is another construct that makes regular expressions more succinct.

3 Automata for Tokenization

In this section, we define *tokenization NFAs*. These automata are minor variants of classical NFAs with ε-transitions that have additional labels specifying the token type id for each state of the automaton.

Definition 4. An ***ε-NFA*** over the alphabet Σ is a tuple $\mathcal{A} = (Q, \delta, \delta_\varepsilon, I, F)$, where Q is the finite *set of states*, $\delta : Q \to \mathsf{List}(\mathcal{P}(\Sigma) \times Q)$ is the *symbol transition function*, $\delta_\varepsilon : Q \to \mathsf{List}(Q)$ is the *ε-transition function*, $I \in \mathsf{List}(Q)$ is the list of *initial states*, and $F \in \mathsf{List}(Q)$ is the list of *final states*.

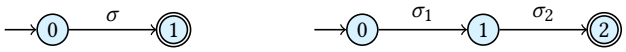
When the order of the states in I and F is irrelevant, we can view them as a set of states, that is, elements of $\mathsf{Set}(Q)$.

Suppose that q is a state of an ε-NFA. We write $q \to^\sigma q'$ to indicate that (σ, q') is in $\delta(q)$. Moreover, for a symbol $a \in \Sigma$, $q \to^a q'$ indicates that there is some $\sigma \subseteq \Sigma$ such that (σ, q') is in $\delta(q)$ and $a \in \sigma$. Similarly, we write $q \to^\varepsilon q'$ to indicate that q' is in $\delta_\varepsilon(q)$.

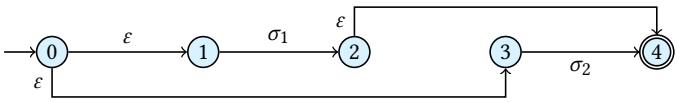
A *path* in $\mathcal{A}$ is a sequence $q_0 \to^{x_0} q_1 \to^{x_1} q_2 \to^{x_2} \cdots \to^{x_{n-1}} q_n$ where (1) each x_i is either a symbol $a \in \Sigma$ or ε, and (2) $q_i \to^{x_i} q_{i+1}$ for every $i = 0, \ldots, n-1$. If the path π_1 ends at state q and the path π_2 starts from q, then $\pi_1 \diamond \pi_2$ is the path that results from concatenating π_1 and π_2 and fusing the states q at the boundary. The *(symbol) label* of a path π, denoted $\mathrm{label}(\pi)$, is the sequence of all alphabet symbols seen in it from left to right. An *accepting path* in $\mathcal{A}$ is a path whose first state is an initial state and whose last state is a final state. We say that $\mathcal{A}$ *accepts* a string $w \in \Sigma^*$ if there is an accepting path in $\mathcal{A}$ whose label is equal to w.

The Thompson construction [Thompson 1968] is a well-known way to construct ε-NFAs that implement regular expressions. The algorithms that we will present in Section 5 make use of Thompson ε-NFAs for tokenization grammars.

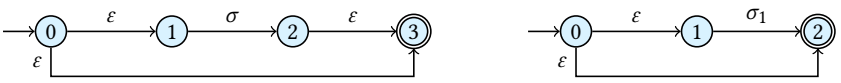
Example 5 (Thompson Construction). The Thompson NFA for the character class σ has states $Q = \{0, 1\}$ with $\Delta(0) = \sigma$. The Thompson NFA for $\sigma_1\sigma_2$ has states $Q = \{0, 1, 2\}$. In particular, note that $\delta(0) = [(\sigma_1, 1)]$, and that $\delta(1) = [(\sigma_2, 2)]$.



The Thompson NFA for the choice $\sigma_1 \mid \sigma_2$ has states $Q = \{0, 1, 2, 3, 4\}$, and we see that $\delta_\varepsilon(0) = [1, 3]$, $\delta(1) = [(\sigma_1, 2)]$, $\delta_\varepsilon(2) = [4]$, and $\delta(3) = [(\sigma_2, 4)]$.

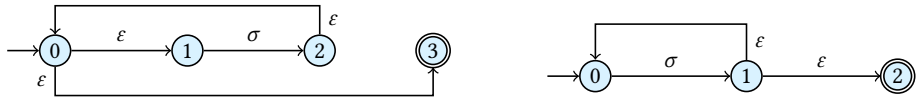


The Thompson NFA for $\sigma? = \sigma \mid \varepsilon$ has states $Q = \{0, 1, 2, 3\}$, for which $\delta_\varepsilon(0) = [1, 3]$, $\delta(1) = [(\sigma, 2)]$, and $\delta_\varepsilon(2) = [3]$ (see diagram below on the left). Since states 2 and 3 are connected with an ε-transition, they can be collapsed. This can streamline the NFA construction for the regex $\sigma?$ (shown below on the right).



The Thompson NFA for σ^* has states $Q = \{0, 1, 2, 3\}$ with $\delta_\varepsilon(0) = [1, 3]$, $\delta(1) = [(\sigma, 2)]$, and $\delta_\varepsilon(2) = [0]$ (see diagram below on the left). For the regex σ^+, we can construct an NFA that has

fewer states than what we would get with the encoding $r^+ = rr^*$ (see diagram below on the right).

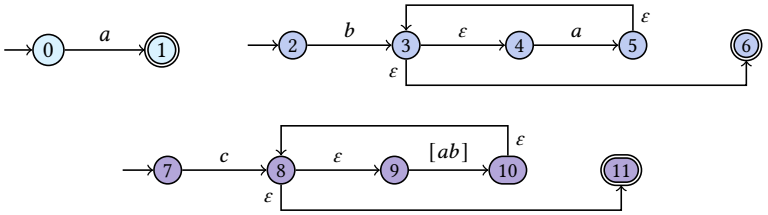


Definition 6 (Epsilon Reachability). Let $\mathcal{A}$ be an ε-NFA and $q \in Q$. We define $\mathrm{R}_\varepsilon(q)$ to be the set of states that are ε-reachable from q.

Definition 7. A ***tokenization NFA*** over the alphabet Σ is a tuple $\mathcal{A} = (Q, \delta, \delta_\varepsilon, I, F, \Lambda)$, where $(Q, \delta, \delta_\varepsilon, I, F)$ is an ε-NFA and $\Lambda : Q \to \mathbb{T}$ is the *token id function*, which satisfies the following properties: (1) $q \to^\sigma q'$ implies that $\Lambda(q) = \Lambda(q')$, (2) $q \to^\varepsilon q'$ implies that $\Lambda(q) = \Lambda(q')$, and (3) if q appears before q' in I then $\Lambda(q) \le \Lambda(q')$.

We define the set $\textsc{Paths}(q, w, i, q')$ of all paths π in $\mathcal{A}$ from q to q' with $\mathrm{label}(\pi) = w[i..]$. Moreover, for a subset of states $S \subseteq Q$, we define $\textsc{Paths}(S, w, i, q') = \bigcup_{q \in S} \textsc{Paths}(q, w, i, q')$. Finally, $\textsc{Reach}(w) = \{q \mid \textsc{Paths}(I, w, i, q) \neq \emptyset \text{ for some } i \text{ with } 0 \le i \le |w|\}$.

Example 8 (Tokenization NFA)**.** Consider the tokenization grammar $\bar{r} = [a, ba^*, c[ab]^*]$. The figure below shows the tokenization NFA for this grammar, where each of the three automata corresponds to one regular expression in the grammar.



Notice that the different token id labels are indicated with different collors: $\Lambda(0) = \Lambda(1) = 0$, $\Lambda(2) = \Lambda(3) = \Lambda(4) = \Lambda(5) = \Lambda(6) = 1$, and $\Lambda(7) = \Lambda(8) = \Lambda(9) = \Lambda(10) = \Lambda(11) = 2$.

The list of initial states is $I = [0, 2, 7]$. The list of final states is $F = [1, 6, 11]$.

4 The Longest Match Problem

This section is a stepping stone towards the first tokenization algorithm of Section 2 (the "token skipping" algorithm of §5.1). Given a tokenization grammar $\bar{r} = [r_0, \ldots, r_{\kappa-1}]$ and an input string w, the *longest match problem* computes at every index $i = 0, \ldots, |w| - 1$ the longest match that ends at the i-th input character as well as the least token type id for this match.

For a regular expression r, define $[\![r]\!]_\mathrm{P} : \Sigma^* \to \mathbb{N}$ as follows: $[\![r]\!]_\mathrm{P}(w)$ is the start position of the longest suffix of w that matches r. More formally, for $w \in \Sigma^*$, we define

$$[\![r]\!]_\mathrm{P}(w) = \min\{i \mid 0 \le i \le |w| \text{ and } w[i..] \in \mathcal{L}(r)\}.$$

We also define $[\![r]\!]_\mathrm{L} : \Sigma^* \to \mathbb{N}$ as follows: $[\![r]\!]_\mathrm{L}(w) = |w| - [\![r]\!]_\mathrm{P}(w)$. This means that $[\![r]\!]_\mathrm{L}(w)$ is the length of the longest suffix of w that matches r. For a tokenization grammar $\bar{r} = [r_0, \ldots, r_{\kappa-1}]$, define $[\![\bar{r}]\!]_\mathrm{I} : \Sigma^* \to \mathbb{T}$ as follows:

$$[\![\bar{r}]\!]_\mathrm{I}(w) = \min\{\beta \mid w[i..] \in \mathcal{L}(r_\beta)\}, \text{ where } i = [\![\bar{r}]\!]_\mathrm{P}(w).$$

That is, $[\![\bar{r}]\!]_\mathrm{I}(w)$ is the least token type index for the longest match.

Note: For all previous semantic functions, the values can be undefined. We have not explicitly considered these cases to reduce the notational clutter. This abuse of notation can be fixed by using

```
// Adds all ε-reachable states that have not been visited yet.
Function AddL(𝒜 : TkNFA(Σ), S : &mut Vect(Thread), added : &mut Set(𝕊), thr : Thread):
    (𝕊 q, ℕ startP) ← thr   // extract the components of the thread
    if q ∈ added then return
    S.push(thr)   // insert the current thread into S
    added.insert(q)   // insert q into the set of "added" states
    for q′ ∈ δ_ε(q) do // loop over the ε-successors of q
        Thread thr′ ← (q′, startP)   // successor thread has same start position
        AddL(𝒜, S, added, thr′)   // process successor thread
// Returns tokens on states that are ε-reachable from the initial states.
Function InitialL(𝒜 : TkNFA(Σ)):
    Vect(Thread) S ← [ ]; Set(𝕊) added ← ∅   // no states have been added yet
    for q ∈ I do // I is the set of initial states of 𝒜
        Thread thr ← (q, 0)   // initial thread, start position 0
        AddL(𝒜, &mut S, &mut added, thr)
    return S
// Returns the new configuration after consuming a symbol from the input.
Function NextL(𝒜 : TkNFA(Σ), S : Vect(Thread), pos : ℕ, a : Σ):
    Vect(Thread) T ← [ ]; Set(𝕊) added ← ∅
    for (q, startP) = S[0], S[1], ..., S[S.len−1] do // process threads in order of priority
        for (σ, q′) ∈ δ(q) with a ∈ σ do // δ is the transition function of 𝒜
            Thread thr ← (q′, startP)   // q′ is an a-successor of q
            AddL(𝒜, &mut T, &mut added, thr)
    for q ∈ I do // I is the set of initial states of 𝒜
        Thread thr ← (q, pos + 1)   // initial thread, start position pos + 1
        AddL(𝒜, &mut T, &mut added, thr)
    return T
// Finds the best match in the configuration.
Function BestMatch(𝒜 : TkNFA(Σ), S : Vect(Thread), pos : ℕ):
    for (q, startP) = S[0], S[1], ..., S[S.len−1] do // process threads in order of priority
        if q ∈ F then // highest-priority match found
            return Some(pos − startP, Λ(q))   // record token length & token id
    return None
// For every position in the input text, find the longest match of r.
Function MatchLongest(r̄ : List(Reg(Σ)), text : Vect(Σ)):
    𝒜 ← Tokenization NFA for r̄ = [r_0, ..., r_{κ−1}]
    Vect(Thread) S ← InitialL(𝒜)   // initial configuration S
    ℕ n ← text.len   // length of input text
    Vect(Option(ℕ × 𝕋)) out ← [None; n]   // output vector, where 𝕋 = {0, ..., κ − 1}
    for pos = 0, 1, ..., n − 1 do // left-to-right pass over the input text
        S ← NextL(𝒜, S, pos, text[pos])   // next configuration & match
        out[pos] ← BestMatch(𝒜, S, pos + 1)
    return out
```

Fig. 1. Algorithm for finding the longest match and the corresponding preferred token id.

the Option type constructor (or by making the functions partial). Notice that either all of $[\![r]\!]_{\mathrm{P}}(w)$, $[\![r]\!]_{\mathrm{L}}(w)$, $[\![r]\!]_{\mathrm{I}}(w)$ are equal to None or all of them are not equal to None.

Let us start with a high-level description of what the main algorithm of this section (i.e., algorithm `MatchLongest` in Fig. 1) computes. Suppose that $\bar{r} = [r_0, \ldots, r_{\kappa-1}]$ is the tokenization grammar and w is the input string. The length of the input string is $n = |w|$. The output is a vector out : Vect(Option($\mathbb{N} \times \mathbb{T}$)), where $\mathbb{T} = \{0, \ldots, \kappa - 1\}$ is the set of all possible token identifiers. For every index $i = 0, \ldots, n-1$, the output vector is equal to $out[i] = (\ell, \beta)$, where $\ell = [\![\bar{r}]\!]_{\mathrm{L}}(w[..i+1])$ and $\beta = [\![\bar{r}]\!]_{\mathrm{I}}(w[..i+1])$. For notational convenience, we are assuming here that (None, None) is the same as None. In other words, the output value $out[i]$ gives the longest match for the nonempty prefix $w[..i+1]$, which has length $i+1$.

The algorithm `MatchLongest` uses NFA simulation. It is more complex than the usual NFA simulation for Boolean matching, since we need to keep track of the length of the longest match and the corresponding token id. In the case of Boolean matching, we simulate an NFA by maintaining a set of active states. We refer to this set of states as a "configuration" of the NFA. We also think that an active state q corresponds to a "thread" of execution. To solve the longest match problem, a thread for a state q needs to carry more information so that we can determine the length of the longest substring (suffix of the prefix $w[..i+1]$ of the input text processed so far) that can lead to state q of the NFA. For Boolean matching, two threads that end up at the same state can be merged into a single thread. For the longest match problem, when merging threads on a state we have to choose the best option, i.e., the longest match. To implement this, a configuration is an ordered collection of threads (implemented as a list of threads) and the algorithm processes the threads in order of priority. Using a list of threads instead of an unordered set has been considered before, e.g., the the matching of PCRE-style regular expressions that use the "leftmost greedy" semantics [Cox 2010]. The longest match problem is more related to the matching of POSIX-style regular expressions, which use the "leftmost longest" semantics. Mamouras et al. [2024] consider both the greedy and POSIX semantics for regex matching and investigate the problem of recognizing when the two semantics differ.

We continue now with some definitions and lemmas that connect the tokenization NFA for a grammar $\bar{r}$ with the functions $[\![r]\!]_{\mathrm{P}}$, $[\![r]\!]_{\mathrm{L}}$, and $[\![r]\!]_{\mathrm{I}}$. A key notion is the configuration $\textsc{Cfg}(w)$ for a string w. As we will see later, computing $\textsc{Cfg}(wa)$ from $\textsc{Cfg}(w)$ involves depth-first search (DFS) with threads being processed in order of priority.

Definition 9 (Configuration). Let $\mathcal{A}$ be a tokenization NFA. If $q \in \textsc{Reach}(w)$, we define the *match position* $\mathsf{MPos}(w, q) = \min\{i \mid 0 \le i \le |w| \text{ and } \textsc{Paths}(I, w, i, q) \neq \emptyset\}$.

A *thread* is a pair (q, i), where $q \in Q$ is a state and $i \in \mathbb{N}$ represents a start position in the input string. Threads are partially ordered as follows: $(q, i) < (q', j)$ iff $i < j$ or $(i = j$ and $\Lambda(q) < \Lambda(q'))$. That is, threads are primarily ordered by start position and then by token id labels.

The *configuration* for w is a partial map $M = \textsc{Cfg}(w)$ whose domain is equal to $\textsc{Reach}(w)$. Moreover, $M(q) = \mathsf{MPos}(w, q)$ for every $q \in \mathrm{dom}(M)$.

Throughout the paper, we use natural numbers for NFA states, that is, $Q \subseteq \mathbb{N}$. We will use the alias $\mathbb{S} = \mathbb{N}$ to denote the set of natural numbers in contexts where they represent control states of NFAs. We also introduce the type $\mathsf{Thread} = \mathbb{S} \times \mathbb{N}$ for threads of the form (q, i), where q is a state and i is a start position in a string.

We can represent a configuration $\textsc{Cfg}(w)$ as a vector $S = [(q_0, i_0), \ldots, (q_{k-1}, i_{k-1})]$: Vect($\mathbb{S} \times \mathbb{N}$) with $M(q_\ell) = i_\ell$, where the states are not duplicated and the threads respect the order that we defined earlier. This means that if $(q_k, i_k) < (q_\ell, i_\ell)$ then $k < \ell$.

We write $S \approx \textsc{Cfg}(w)$ to denote that the vector S is a representation of the configuration $\textsc{Cfg}(w)$. Notice that a configuration can have many different representations, since threads are partially ordered (which means that incomparable threads can be given in any order).

Lemma 10 (Init & Step). Let $\mathcal{A}$ be a tokenization NFA, $w \in \Sigma^*$, and $a \in \Sigma$. The following hold:

(1) If $q \in \textsc{Reach}(\varepsilon) = \mathrm{R}_\varepsilon(q_{in})$, then $\mathsf{MPos}(\varepsilon, q) = 0$.
(2) For every $q'' \in \textsc{Reach}(w)$, we have that $\mathsf{MPos}(wa, q'') = |wa|$ or $\mathsf{MPos}(wa, q'') = \mathsf{MPos}(w, q)$ for some states q, q' with $q \to^a q'$ and $q'' \in \mathrm{R}_\varepsilon(q')$.

A configuration can also be viewed as a set of threads (q, i) with no duplicate states (i.e., no state appears more than once in the configuration). For a configuration S, we define

$$\textsc{BestP}(S) = \min\{i \mid (q, i) \in S \cap (F \times \mathbb{N})\} \text{ and}$$
$$\textsc{BestI}(S) = \min\{\Lambda(q) \mid (q, \textsc{BestP}(S)) \in S \cap (F \times \mathbb{N})\}.$$

Let S be a configuration and $n \in \mathbb{N}$, where n is meant to correspond to the length of the input that has been processed. If S does not contain a final state, we define $\textsc{BestL}(S, n) = \text{None}$. If S contains a final state, then we define $\textsc{BestL}(S, n) = n - \textsc{BestP}(S)$.

Lemma 11. Let $\bar{r} = [r_0, \ldots, r_{k-1}]$ and $\mathcal{A}$ be the TkNFA for $\bar{r}$. For a string $w \in \Sigma^*$, we have that

$$[\![\bar{r}]\!]_{\mathrm{P}}(w) = \textsc{BestP}(S) \qquad [\![\bar{r}]\!]_{\mathrm{L}}(w) = \textsc{BestL}(S, |w|) \qquad [\![\bar{r}]\!]_{\mathrm{I}}(w) = \textsc{BestI}(S),$$

where $S = \textsc{Cfg}(w)$.

Lemma 11 says that the tokenization NFA for $\bar{r}$ can be used to implement the denotations $[\![\bar{r}]\!]_{\mathrm{P}}$, $[\![\bar{r}]\!]_{\mathrm{L}}$, and $[\![\bar{r}]\!]_{\mathrm{I}}$.

Tokenization NFA Execution for Finding the Longest Match (Fig. 1): In order to implement the "longest match" semantics given by $[\![\bar{r}]\!]_{\mathrm{P}}$, $[\![\bar{r}]\!]_{\mathrm{L}}$ and $[\![\bar{r}]\!]_{\mathrm{I}}$, it is not sufficient to simulate the NFA $\mathcal{A}_{\bar{r}}$ for $\bar{r}$ in the usual way, where we maintain the (unordered) set of active states. This is because we prefer the longest match and we also need to record the start position and/or the length of the longest match. We also need to indicate the preferred token id that corresponds to the longest match.

For this reason, the algorithm of Fig. 1 maintains a configuration that is a (linearly ordered) list/vector of threads, where a thread is an active NFA state together with some additional information. In this case, the additional information is the start position in the string for the NFA execution path that gave rise to the thread. More specifically, a configuration

$$S = [(q_0, i_0), \ldots, (q_{k-1}, i_{k-1})] : \mathsf{Vect}(\mathbb{S} \times \mathbb{N})$$

respects the *priority order* of the threads: high-priority threads come before low-priority threads. More formally, the *priority* of a thread (q, i) is the pair $(i, \Lambda(q)) : \mathbb{N} \times \mathbb{T}$, where i is a position in the string and $\Lambda(q)$ is the token type id for q. Priorities are ordered as follows: $(i, \beta) < (j, \gamma)$ iff $i < j$ or $(i = j$ and $\beta < \gamma)$. When $(i, \beta) < (j, \gamma)$, the priority (i, β) is higher than (j, γ). So, for the configuration S, the following should hold:

$$\underbrace{(i_k, \Lambda(q_k))}_{\text{priority of } (q_k, i_k)} < \underbrace{(i_\ell, \Lambda(q_\ell))}_{\text{priority of } (q_\ell, i_\ell)} \implies k < \ell,$$

which means that the thread (q_k, i_k) precedes (q_ℓ, i_ℓ) in S.

The function `InitialL` of Fig. 1, whose output type is $\mathsf{Vect}(\mathsf{Thread}) = \mathsf{Vect}(\mathbb{S} \times \mathbb{N})$, creates the initial configuration $\textsc{Cfg}(\varepsilon)$. All threads in the initial configuration have the same start position, which is equal to 0. Notice that the main loop in `InitialL` goes over the initial states $q \in I$ in order of token type id. This order is essential for maintaining the key ordering invariant for configurations.

One important observation is that any *successor* of a thread (q, i) *has the same priority* as the thread (q, i). This means that the computation of the NFA should always consider threads in order of priority. This property about the priorities of thread successors can be seen by considering an ε-transition $(q, i) \rightarrow^{\varepsilon} (q', i)$ and a letter transition $(q, i) \rightarrow^{a} (q', i)$. In both cases, the start position remains the same. Moreover, we know from Definition 7 that $q \rightarrow^{\varepsilon} q'$ or $q \rightarrow^{a} q'$ implies that $\Lambda(q) = \Lambda(q')$.

The function `AddL` of Fig. 1 receives the following arguments: a thread *thr* : Thread, a pointer S : &mut Vect(Thread) to a vector of threads, and a pointer *added* : &mut Set($\mathbb{S}$) to a set of states. The purpose of *added* is to keep track of the visited NFA states, so that they are not processed more than once. The vector S is meant to maintain an appropriately ordered list of threads for the next step. The function `AddL` adds all states that are ε-reachable from *thr* and that have not been visited before. It is essentially an implementation of depth-first search (DFS) that additionally keeps track of start position information.

The function `NextL` of Fig. 1, which has output type Vect(Thread), performs the main computation step of the automaton. It receives as arguments the current configuration S : Vect(Thread) (which should satisfy the ordering invariant discussed earlier), the current position $pos \in \mathbb{N}$ in the string, and an input symbol $a \in \Sigma$. It returns the next configuration, i.e., after consuming the input symbol a. The correctness of this function hinges on processing the threads of S *in order of priority*. For a thread (q, i) with $q \rightarrow^{a} q'$, we create the thread (q', i) and we invoke `AddL` with (q', i) in order to visit all (previously un-visited) ε-reachable states. After all threads have been processed, we continue to *restart* the automaton by adding all initial states with start position $pos + 1$. This needs to be done, because we want to consider the paths from all start positions in the input string.

The function `BestMatch` of Fig. 1 has output type Option($\mathbb{N} \times \mathbb{T}$). It returns the best match (match length & corresponding token type id) that corresponds to a given configuration S : Vect(Thread). The precondition for this function (as with all other functions) is that the configuration S should satisfy the ordering invariant discussed earlier. Then, we can find the best match by traversing the threads in the given order. When we find the first final state, then the corresponding thread gives us the best match. If no final state is found, then we return None.

The function `MatchLongest` of Fig. 1 is the top-level function for executing the automaton and computing the "longest match" semantics. The first line constructs the tokenization NFA $\mathcal{A}$ from the input tokenization grammar $\bar{r} = [r_0, \ldots, r_{\kappa+1}]$. The initial configuration is constructed using `InitialL`. The main loop performs a left-to-right traversal over the input text. At every step, the function `NextL` is used to construct the next configuration, and the function `BestMatch` is used to compute the best match for the current position.

Proposition 12 (Correctness of `MatchLongest`)**.** Let $\bar{r} = [r_0, \ldots, r_{\kappa-1}]$ and out = `MatchLongest`$(\bar{r}, w)$ for the algorithm of Fig. 1. For every $i = 0, \ldots, |w| - 1$, we have that $out[i] = (\ell, \beta)$, where $\ell = [\![\bar{r}]\!]_{\mathrm{L}}(w[..i+1])$ and $\beta = [\![\bar{r}]\!]_{\mathrm{I}}(w[..i+1])$.

Proof. Let $\mathcal{A}$ be the tokenization NFA for $\bar{r}$. The following hold:

(1) `InitialL`$(\mathcal{A}) \approx \textsc{Cfg}(\varepsilon)$.
(2) For every $w \in \Sigma^*$ and $a \in \Sigma$, if $S \approx \textsc{Cfg}(w)$ then `NextL`$(\mathcal{A}, S, |w|, a) \approx \textsc{Cfg}(wa)$.

The correctness of the above properties relies on the correctness of depth-first search (DFS algorithm `AddL` for ε-closure), Lemma 10, and the ordering invariant for configurations that the algorithm `MatchLongest` satisfies (i.e., threads appear in order of priority).

Consider now the i-th repetition of the main loop. We will discuss the correctness of the output. The algorithm computes $out[i]$ = `BestMatch`$(\mathcal{A}, S, i+1)$, where $S \approx \textsc{Cfg}(w[..i+1])$. From the code of `BestMatch` we easily see that `BestMatch`$(\mathcal{A}, S, i+1) = (\ell, \beta)$, where $\ell = \textsc{BestL}(S, i+1)$

index i		0	1	2	3	4	5	6	7	8	9	10
string w		a	b	a	a	b	a	c	a	b	a	a
config.	$(0,0)$	$(1,0)$	$(3,1)$	$(5,1)$	$(5,1)$	$(3,4)$	$(5,4)$	$(8,6)$	$(10,6)$	$(10,6)$	$(10,6)$	$(10,6)$
	$(2,0)$	$(0,1)$	$(4,1)$	$(3,1)$	$(3,1)$	$(4,4)$	$(3,4)$	$(9,6)$	$(8,6)$	$(8,6)$	$(8,6)$	$(8,6)$
	$(7,0)$	$(2,1)$	$(6,1)$	$(4,1)$	$(4,1)$	$(6,4)$	$(4,4)$	$(11,6)$	$(9,6)$	$(9,6)$	$(9,6)$	$(9,6)$
		$(7,1)$	$(0,2)$	$(6,1)$	$(6,1)$	$(0,5)$	$(6,4)$	$(0,7)$	$(11,6)$	$(11,6)$	$(11,6)$	$(11,6)$
			$(2,2)$	$(1,2)$	$(1,3)$	$(2,5)$	$(1,5)$	$(2,7)$	$(1,7)$	$(3,8)$	$(5,8)$	$(5,8)$
			$(7,2)$	$(0,3)$	$(0,4)$	$(7,5)$	$(0,6)$	$(7,7)$	$(0,8)$	$(4,8)$	$(3,8)$	$(3,8)$
				$(2,3)$	$(2,4)$		$(2,6)$		$(2,8)$	$(6,8)$	$(4,8)$	$(4,8)$
				$(7,3)$	$(7,4)$		$(7,6)$		$(7,8)$	$(0,9)$	$(6,8)$	$(6,8)$
										$(2,9)$	$(1,9)$	$(1,10)$
										$(7,9)$	$(0,10)$	$(0,11)$
											$(2,10)$	$(2,11)$
											$(7,10)$	$(7,11)$
$[\![\bar{r}]\!]_{\mathrm{L}}(w[..i+1])$		1	1	2	3	1	2	1	2	3	4	5
$[\![\bar{r}]\!]_{\mathrm{I}}(w[..i+1])$		0	1	1	1	1	1	2	2	2	2	2

Fig. 2. Execution of algorithm `MatchLongest` of Fig. 1 for the tokenization grammar $\bar{r} = [a, ba^*, c[ab]^*]$.

and $\beta = \textsc{BestI}(S)$. Since $|w[..i+1]| = i+1$, we get from Lemma 11 that $\ell = \textsc{BestL}(S, i+1) = [\![\bar{r}]\!]_{\mathrm{L}}(w[..i+1])$ and $\beta = [\![\bar{r}]\!]_{\mathrm{I}}(w[..i+1])$. □

Complexity of Algorithm. Let n be the length of the input text and m be the size of the tokenization grammar. The tokenization NFA $\mathcal{A}$ has size m, because we use Thompson's construction. Since we use an efficient reachability algorithm for ε-closure (depth-first search, see `AddL`), both `InitialL` and `NextL` need time $O(m)$. So, in total, the running time is $O(mn)$. The work space needed is $O(m)$ for storing the configuration S.

Example 13 (Execution of `MatchLongest`)**.** Let $\bar{r} = [a, ba^*, c[ab]^*]$ and $w = abaabacabaa$. The tokenization NFA for $\bar{r}$ is shown in Example 8. Fig. 2 shows the execution of `MatchLongest` on the input string w.

First, the initial configuration consists of the initial threads $(0,0)$, $(2,0)$, $(7,0)$, where the start position is 0. After consuming the letter $w[0] = a$, the thread $(0,0)$ is moved forward and becomes $(1,0)$. However, the other two threads $(2,0)$ and $(7,0)$ are dropped because the states 2 and 7 are guarded by incompatible letters, as described by line 18 in Fig. 1. The other three threads shown at index 0 in Fig. 2, which are $(0,1)$, $(2,1)$, and $(7,1)$, come from reinitializing the automata, but at string position 1. Now, at this stage, we have already found a match because the state 1 is an accepting state. Recall that the notation $[\![r]\!]_{\mathrm{L}}(w)$ indicates the length of the longest suffix of w that matches r. It is easy to see that at this stage, $[\![\bar{r}]\!]_{\mathrm{L}}(w[..i+1]) = [\![\bar{r}]\!]_{\mathrm{L}}(w[..1]) = [\![\bar{r}]\!]_{\mathrm{L}}(a) = 1$ because the longest suffix of a that matches r is a itself, which is of length 1. Also, recall that the notation $[\![r]\!]_{\mathrm{I}}(w)$ indicates the least token type index for the aforementioned longest match. Thus, we see that at this stage, $[\![\bar{r}]\!]_{\mathrm{I}}(w[..i+1]) = [\![\bar{r}]\!]_{\mathrm{I}}(a) = 0$ because the regular expression a, which has token type index 0 in $\bar{r}$, corresponds to this match.

Similarly, in the next step, when the algorithm consumes the letter $w[1] = b$, the threads $(1,0)$, $(0,1)$, $(7,1)$ are dropped due to incompatible transitions. However, the thread $(2,1)$ gets moved forward and spawns $(3,1)$, $(4,1)$, and $(6,1)$. The other threads shown at index 1 in Fig. 2, as before, come from reinitializing the automata at string index 2. At this stage, $[\![\bar{r}]\!]_{\mathrm{L}}(w[..i+1]) =$

```
   // Input: tokenization grammar r̄ = [r_0, ..., r_{κ-1}] and string text
1  Function TokenizeBacktrack(r̄ : List(Reg(Σ)), text : Vect(Σ)):
2      A ← Tokenization DFA for r̄ = [r_0, ..., r_{κ-1}]
3      Vect(Σ* × T) out ← [ ]   // output vector, where T = {0, ..., κ − 1}
4      ℕ startP ← 0   // start position for current token
5      while startP < text.len do // left-to-right pass with backtracking
6          Option(ℕ × T) tk ← None   // best token found so far in this round
7          𝕊 q ← initial state of A
8          ℕ pos ← startP
9          while pos < text.len do // left-to-right pass over the input text
10             q ← δ(q, text[pos])   // δ is the transition function of A
11             if q is final then
12                 tk ← Some(pos + 1 − startP, Λ(q))   // Λ(q): preferred token id
13             if q is a failure state then break
14             pos ← pos + 1
15         if let Some(ℓ, β) = tk then
16             out.push(text[startP..startP + ℓ], β)   // store token in the output list
17             startP ← startP + ℓ   // continue after the token
18         else break
19     return out
```

Fig. 3. The standard DFA-based backtracking algorithm for tokenization (see flex [Vern Paxson 1987]).

$[\![\bar{r}]\!]_{\mathrm{L}}(ab) = 1$ because the longest suffix of ab that matches $\bar{r}$ is b, which is of length 1. Also, $[\![\bar{r}]\!]_{\mathrm{I}}(w[..i+1]) = [\![\bar{r}]\!]_{\mathrm{I}}(ab) = 1$ because the corresponding regular expression ba^* has index 1 in $\bar{r}$.

At index 2 in Fig. 2, the threads $(3, 1)$, $(6, 1)$, $(2, 2)$, and $(7, 2)$ are dropped due to incompatible transitions. The thread $(4, 1)$ spawns $(5, 1)$, $(3, 1)$, $(4, 1)$, and $(6, 1)$ by moving $(4, 1)$ forward and taking the ε-closure. Finally, the thread $(0, 2)$ gets moved forward and spawns $(1, 2)$. The new threads $(0, 3)$, $(2, 3)$, and $(7, 3)$, as before, are spawned due to initialization of the automata at string index 3. At this stage, $[\![\bar{r}]\!]_{\mathrm{L}}(w[..i+1]) = [\![\bar{r}]\!]_{\mathrm{L}}(aba) = 2$ because the longest suffix of aba that matches $\bar{r}$ is ba, which is of length 2. This match corresponds to the regular expression ba^*, which as index 1 in $\bar{r}$, so $[\![\bar{r}]\!]_{\mathrm{I}}(w[..i+1]) = [\![\bar{r}]\!]_{\mathrm{I}}(aba) = 1$.

The rest of the algorithm execution can be understood using the same line of reasoning.

5 Efficient Tokenization Algorithms

The efficient tokenization algorithms that we will present in this section address the main shortcoming of flex's algorithm, which is the quadratic (in the length of the input text) running time. See Fig. 3 for flex's algorithm, presented in a way that solves the uniform tokenization problem. The outer while loop performs one repetition for every token that is identified. The inner while loop reads the input symbols from left to right starting with position *startP* until the longest token for the suffix $text[startP..]$ is found. As discussed earlier, the source of inefficiency in this algorithm is the fact that the algorithm *backtracks* (because the inner index *pos* can move ahead of the token). This means that it can read parts of the input several times.

As mentioned in the introduction, Reps [1998] has shown how to eliminate the time inefficiency of backtracking by adding a memoization table to the algorithm. This table records for every pair (q, i), where q is a DFA state and i is a position in the string, whether (q, i) is known to lead to failure. Using this information, the algorithm of [Reps 1998] avoids the repetition of work. The problem now is that this memoization table can be quite large, since it requires $O(Mn)$ bits of

memory to store, where M is the DFA size. For example, for input text of length $n = 4\,\text{GB}$ and $M = 100$ (these sizes are realistic for JSON data analysis), the memory requirements would be 50 GB of memory, which can exceed the available system memory in many cases.

In order to completely eliminate backtracking, we notice that we need information about the future input. This information can be pre-computed by first performing a right-to-left pass over the input text, before we identify the tokens. The algorithms of §5.1 and §5.2 explore two different pre-computation approaches. Our algorithms retain the pleasant time-complexity property of [Reps 1998] (i.e., linear time in the length of the input text), but require less memory.

5.1 The "Token Skipping" Approach

Our first efficient tokenization algorithm uses the "longest match" algorithm of Section 4. The idea behind the algorithm is to process the input text *in reverse* first, in order to obtain information about the future tokens. Suppose that some initial tokens have been identified, which altogether have consumed i characters of the input string w. This means that it remains to tokenize the suffix $w[i..]$. The next token (if one exists) starts at position i. If, somehow, we knew the length of the next token, then it would be very easy to find the first token in the remaining string. Suppose that we have this information already available, a data structure $tape : \mathsf{Vect}(\mathbb{N})$ so that $tape[i] = \ell$. The interpretation of $tape$ is the following: $tape[i]$ is equal to the *length* of the maximal-munch token for the suffix $w[i..]$. Using $tape$, we can easily tokenize the entire string as follows:

(1) The first token is $w[0..\ell_1]$, where $\ell_1 = tape[0]$.
(2) The second token is $w[\ell_1..\ell_1 + \ell_2]$, where $\ell_2 = tape[\ell_1]$.
(3) The third token is $w[\ell_1 + \ell_2..\ell_1 + \ell_2 + \ell_3]$, where $\ell_3 = tape[\ell_1 + \ell_2]$.

This process continues until the entire string is tokenized.

We need to explain now how the data structure $tape$ of the previous paragraph is related to the "longest match" algorithm of Section 4. According to the maximal-munch semantics, it should be that $tape[i] = next[i] - i$, where

$$next[i] = \max\{j \mid i \leq j \leq |w| \text{ and } w[i..j] \in \mathcal{L}(\bar{r})\}.$$

If we think of traversing the string from right to left, we can see that

$$tape[i] = [\![\mathsf{rev}(\bar{r})]\!]_{\mathrm{P}}(\mathsf{rev}(w[i..])).$$

In other words, computing $tape$ is the same as computing the "longest match" semantics over the reversed input string $\mathsf{rev}(w)$ using the reversed tokenization grammar $\mathsf{rev}(\bar{r})$.

The function `ReverseMatch` of Fig. 4 has output type $\mathsf{Vect}(\mathsf{Option}(\mathbb{N} \times \mathbb{T}))$. Notice that it is essentially the same as the function `MatchLongest` of Fig. 1 with the difference being that it performs a right-to-left traversal of the input string *text* and it uses the *reversed* tokenization grammar $\mathsf{rev}(\bar{r})$ when constructing the tokenization NFA.

Theorem 14 (Correctness of Token Skipping). Let $\bar{r} = [r_0, \ldots, r_{\kappa-1}]$ be a tokenization grammar and $w \in \Sigma^*$. For the algorithm of Fig. 4, we have that

$$out = \texttt{Tokenize1}(\bar{r}, w) = \mathsf{tokens}(\bar{r})(w).$$

Proof. The proof proceeds by induction on $|w| - pos$, that is, the length of the input string suffix that remains to be processed. We have to establish that in one repetition of the main loop of `Tokenize1`, we identify the first most preferred token (according to the maximal-munch semantics) of the suffix $u = w[pos..]$. If there is no nonempty token, then the algorithm terminates, which agrees with the tokenization semantics of Definition 1. The interesting case is when $\mathsf{token}(\bar{r})(u) = \mathrm{Some}(v, \beta)$, where v is a prefix of u and $\beta \in \{0, \ldots, \kappa - 1\}$. The token is correctly identified exactly when $tape[pos] = \mathrm{Some}(|v|, \beta)$.

```
// For every position i in the input text, find the longest match [i, j] of rev(r).
Function ReverseMatch(r̄ : List(Reg(Σ)), text : Vect(Σ)):
    A_rev ← Tokenization NFA for rev(r̄) = [rev(r_0), ..., rev(r_{κ−1})]
    Vect(Thread) S ← InitialL(A_rev)   // initial configuration S
    ℕ n ← text.len   // length of input text
    Vect(Option(ℕ × 𝕋)) tape ← [None; n]   // output vector, where 𝕋 = {0, ..., κ − 1}
    for i = 0, 1, ..., n − 1 do // right-to-left pass over the input text
        ℕ pos ← n − 1 − i   // position in the input text
        S ← NextL(A_rev, S, i, text[pos])   // next configuration
        tape[pos] ← BestMatch(A_rev, S, i + 1)
    return tape
// Input: tokenization grammar r̄ = [r_0, ..., r_{κ−1}] and string text
Function Tokenize1(r̄ : List(Reg(Σ)), text : Vect(Σ)):
    Vect(Option(ℕ × 𝕋)) tape ← ReverseMatch(r̄, text)
    Vect(Σ* × 𝕋) out ← [ ]   // output vector, where 𝕋 = {0, ..., κ − 1}
    ℕ pos ← 0
    while pos < text.len do // left-to-right pass over tape
        if let Some(ℓ, β) = tape[pos] then
            if ℓ = 0 then break   // there is no further nonempty token
            out.push(text[pos..pos + ℓ], β)   // store token in the output list
            pos ← pos + ℓ   // skip over the token ("token skipping")
        else break
    return out
```

Fig. 4. TokenSkip: Efficient tokenization algorithm using the "token skipping" approach.

Now, we notice that $|v| = [\![\mathrm{rev}(\bar{r})]\!]_{\mathrm{L}}(\mathrm{rev}(u))$ and $\beta = [\![\mathrm{rev}(\bar{r})]\!]_{\mathrm{I}}(\mathrm{rev}(u))$, according to the "longest match" semantics of Section 4. Since *tape* is computed by `ReverseMatch`, which is the same as `MatchLongest` on $\mathrm{rev}(\bar{r})$ and $\mathrm{rev}(w)$, Proposition 12 concludes the proof. □

Complexity of Tokenization with Token Skipping. Let n be the length of the input text and m be the size of the tokenization grammar. The tokenization NFA $\mathcal{A}_{\mathrm{rev}}$ has size m, because we use Thompson's construction on the reversed grammar $\mathrm{rev}(\bar{r})$, which has the same size as $\bar{r}$. The function `ReverseMatch` is used once in `Tokenize1` and its time complexity is $O(mn)$, as we have already discussed in Section 4. The main loop of `Tokenize1` performs a traversal of *tape* (it may skip some entries), which is $O(n)$ work. So, the total running time is $O(mn)$. The work space needed is $O(m + n)$ memory locations for storing the configuration S (see `ReverseMatch`) and the vector *tape*.

Example 15 (Token Skipping)**.** Let $\bar{r} = [a, a^*b, [ab]^*c]$ and $w = aabacabaaba$.

index	0	1	2	3	4	5	6	7	8	9	10
string	a	a	b	a	c	a	b	a	a	b	a
tape.token_length	5	4	3	2	1	2	1	3	2	1	1
tape.token_id	2	2	2	2	2	1	1	1	1	1	0

The main loop of `Tokenize1` will execute 4 times. In the first repetition, we have $pos = 0$ and $tape[pos] = \mathrm{Some}(5, 2)$. This means that we identify the token $(w[0..5], 2) = (aabac, 2)$ and we increment *pos* by 5. In the second repetition, we have $pos = 5$ and $tape[pos] = \mathrm{Some}(2, 1)$. We identify the token $(w[5..7], 1) = (ab, 1)$ and we increment *pos* by 2. In the third repetition, we have

```
Function Initial(𝒜 : NFA(Σ)):
    Return the set of NFA states that are ε-reachable from the initial states.

Function Next(𝒜 : NFA(Σ), S : Set(𝕊), a : Σ):
    Let S′ be the set of states obtained from S by transitioning on a.
    Return the set of states that are ε-reachable from S′.

// For every position, compute the set of states that can lead to a longer token.
1  Function ReverseMatch(𝒜 : TkNFA(Σ), text : Vect(Σ)):
2      𝒜_rev ← reverse of 𝒜
3      Set(𝕊) S ← Initial(𝒜_rev)   // initial configuration S
4      ℕ n ← text.len   // length of input text
5      Vect(Set(𝕊)) oracle ← [∅; n + 1]   // output vector
6      for pos = n − 1, ..., 1, 0 do // right-to-left pass over the input text
7          S ← Next(𝒜_rev, S, text[pos])   // next configuration
8          oracle[pos] ← S   // store configuration in the output vector
9          S ← S ∪ Initial(𝒜_rev)   // restart the reverse automaton
10     return oracle

// Input: tokenization grammar r̄ = [r_0, ..., r_{κ−1}] and string text
11 Function Tokenize2(r̄ : List(Reg(Σ)), text : Vect(Σ)):
12     𝒜 ← Tokenization NFA for r̄ = [r_0, ..., r_{κ−1}]
13     Set(𝕊) S ← Initial(𝒜)   // initial configuration S
14     Vect(Set(𝕊)) oracle ← ReverseMatch(𝒜, text)
15     Vect(Σ* × 𝕋) out ← [ ]   // output vector, where 𝕋 = {0, ..., κ − 1}
16     ℕ startP ← 0   // start position for current token
17     for pos = 0, 1, ..., text.len − 1 do // left-to-right pass over the input text
18         S ← Next(𝒜, S, text[pos])
19         if S ∩ F ≠ ∅ then // final state reached
20             if S ∩ oracle[pos + 1] = ∅ then // the token cannot be extended
21                 Σ* tk ← text[startP..pos + 1]   // longest token
22                 𝕋 β ← min{Λ(q) | q ∈ S ∩ F}   // token type id
23                 out.push(tk, β)   // store token in the output list
24                 startP ← pos + 1   // start position for next token
25                 S ← Initial(𝒜)   // initial configuration
26     return out
```

Fig. 5. ExtOracleNoOpt: Efficient tokenization algorithm using the "token extension oracle" approach.

$pos = 7$ and $tape[pos] = \text{Some}(3, 1)$. So, the token is $(w[7..10], 1) = (aab, 1)$ and pos is incremented by 3. Finally, in the fourth repetition, we have $pos = 10$ and $tape[pos] = \text{Some}(1, 0)$. The last token is $(w[10..11], 0) = (a, 0)$ and pos is incremented by 1. The loop terminates because $pos = |w| = 11$ and the output is $out = [(aabac, 2), (ab, 1), (aab, 1), (a, 0)]$.

5.2 The "Token Extension Oracle" Approach

Our second efficient tokenization algorithm, shown in Fig. 5, is based on the idea of having a "token extension oracle" that tells us whether an identified token can be further extended or not. In the usual backtracking algorithm for tokenization, when we identify a token (final state reached) we have to continue exploring the input for the possibility that the token can be extended to a longer one. This exploration will ultimately fail or explore the entire string. At this point, the algorithm backtracks to the end position of the longest token that was found in order to continue.

Our algorithm of Fig. 5 is based on the simulation of ε-NFAs. For this algorithm, the notion of configuration is the classical one: a configuration is a set of active NFA states. We can see in the main loop of `Tokenize2` that the oracle is used when the current configuration S contains a final state, i.e., $S \cap F \neq \emptyset$. When this happens, we want to check whether the token that we have just identified can be extended to a longer one. This happens when there exists a state $q \in S$ such that there is a path from q to some final state using more input from the remaining string $text[pos+1..]$. The entry $oracle[pos+1]$ of the "oracle tape" $oracle : \mathrm{Vect}(\mathrm{Set}(\mathbb{S}))$ contains all such states q that witness a longer token.

We can compute the "oracle tape" by performing a right-to-left pass over the input string. If $\mathcal{A}$ is the tokenization NFA for the input grammar $\bar{r}$, we consider the *reverse* ε-NFA and we simulate its execution on $\mathrm{rev}(text)$. At every step, we record the configuration of the automaton. We also restart the automaton at every step, because we want to consider all possible end positions for tokens.

Theorem 16 (Correctness of Extension Oracle). Let $\bar{r} = [r_0, \ldots, r_{\kappa-1}]$ be a tokenization grammar and $w \in \Sigma^*$. For the algorithm of Fig. 5, we have that

$$out = \texttt{Tokenize2}(\bar{r}, w) = \mathsf{tokens}(\bar{r})(w).$$

Proof. The proof proceeds by induction on $|w| - pos$, that is, the length of the input string suffix that remains to be processed. If there is no further token to be found, then the algorithm will never construct a configuration S that intersects F, which means that the algorithm behaves correctly. The interesting case is when there is another token to be found, so we will consider this case from now on. The main loop satisfies the invariant $startP \leq pos$ and also that $w[startP..pos]$ is a prefix of the next token. When the test $S \cap F \neq \emptyset$ succeeds, we know that $w[startP..pos+1]$ is a token. Assuming that the oracle tape contains the appropriate set of states, the test $S \cap oracle[pos+1] = \emptyset$ succeeds exactly when the token cannot be further extended, so it should be added to the output according to the maximal-munch semantics.

According to the previous paragraph, correctness of `Tokenize2` relies on *oracle* satisfying the following property:

$$oracle[pos] = \{q \in Q \mid \text{there is } j > pos \text{ and a path } \pi \text{ from } q \text{ to } F \text{ with label } w[pos..j]\}.$$

It remains to see that `ReverseMatch` computes *oracle* as specified above. Indeed, this is the case, because `ReverseMatch` reverses the tokenization NFA $\mathcal{A}$ to obtain $\mathcal{A}_{\mathsf{rev}}$ and then uses classical ε-NFA simulation to compute states that are reachable from the initial states of $\mathcal{A}_{\mathsf{rev}}$ (which are the final states of $\mathcal{A}$). □

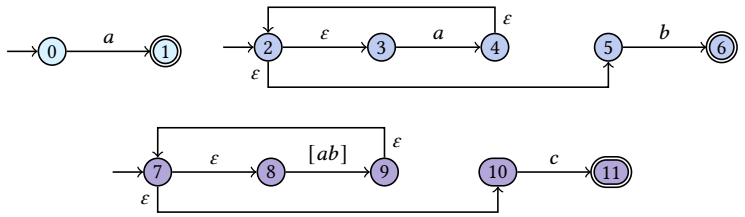
Complexity of Tokenization with Extension Oracle. Let n be the length of the input text and m be the size of the tokenization grammar. The tokenization NFA $\mathcal{A}$ has size $O(m)$, because we use Thompson's construction. Moreover, the reversed NFA $\mathcal{A}_{\mathsf{rev}}$ also has size $O(m)$. The function `ReverseMatch` performs a standard simulation of ε-NFA execution using sets of active states. So, the running time for this function is $O(mn)$, using any efficient reachability algorithm for computing the ε-closure (e.g., depth-first search or breadth-first search). The main loop of `Tokenize2` performs a standard simulation of the ε-NFA $\mathcal{A}$, which needs time $O(mn)$. All the tests involving sets of states can be efficiently performed in $O(m)$ time using an appropriate representation of state sets. The work space used by the algorithm is dominated by the oracle tape, which needs $O(mn)$ bits of memory.

Example 17 (The "Token Extension Oracle" Algorithm)**.** Consider the tokenization grammar $\bar{r} = [a, a^*b, [ab]^*c]$ and the input string $w = aabacabaaba$. We will describe the execution of the

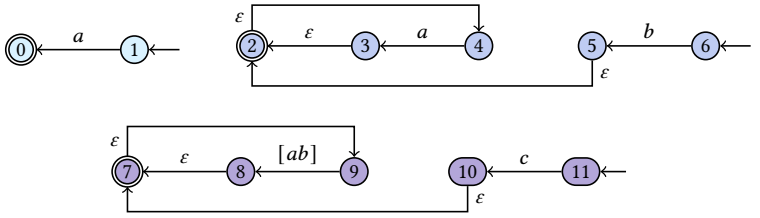
index		0	1	2	3	4	5	6	7	8	9	10
string		a	a	b	a	c	a	b	a	a	b	a
oracle		0	0	2	0	7	0	2	0	0	2	0
		2	2	4	7	9	2	4	2	2	4	
		3	3	5	8	10	3	5	3	3	5	
		4	4	7	9		4		4	4		
		7	7	8								
		8	8	9								
		9	9									
config. S	0	1	2	6	7	11	1	6	1	2	6	1
	2	2	3	7	8		2	7	2	3	7	2
	7	3	4	8	9		3	8	3	4	8	3
		4	5	9	10		4	9	4	5	9	4
		5	7	10			5	10	5	7	10	5
		7	8				7		7	8		7
		8	9				8		8	9		8
		9	10				9		9	10		9
		10					10		10			10
token length						5		2			3	1
token id						2		1			1	0

Fig. 6. Execution of algorithm `Tokenize2` of Fig. 5 for $\bar{r} = [a, a^*b, [ab]^*c]$.

tokenization algorithm of Fig. 5. The tokenization NFA $\mathcal{A}$ for $\bar{r}$ is shown below:



The reverse automaton $\mathcal{A}_{\text{rev}}$ of the tokenization NFA is shown below:



The main aspects of the execution of the algorithm are shown in Fig. 6.

First, the row *oracle* is computed with a right-to-left pass over the input string using the reverse automaton $\mathcal{A}_{\text{rev}}$. Then, we perform a left-to-right pass over the input string using the forward automaton $\mathcal{A}$. This pass corresponds to the row "configuration S in the figure.

The first token is identified after processing the symbol c at index 4. The state 11 is final, but we know that the token cannot be extended because {11} does not intersect *oracle*[5].

5.3 Performance Optimization for the "Extension Oracle" Approach

The algorithms of §5.1 and §5.2 are based on nondeterministic automata, which means that they avoid the exponential blowup (in the worst case) that is associated with NFA-to-DFA constructions. For practical tokenization grammars, it is typically the case that the corresponding DFA is not substantially larger than the NFA for the grammar. When this is the case, DFA execution is preferable to NFA simulation, because each DFA step needs $O(1)$ time, whereas each NFA simulation step needs $O(m)$ operations.

For this reason, we refine the "token extension oracle" algorithm of Fig. 5 to use DFAs instead of NFAs when their size is not too large. We call this algorithm ExtOracleOpt. If the DFAs are large, then the ExtOracleOpt algorithm will use NFAs and behave exactly as ExtOracleNoOpt (Fig. 5). So, ExtOracleOpt can be understood as a hybrid DFA/NFA algorithm. This means that the following are constructed before the actual tokenization takes place:

(1) the DFA $\mathcal{A}_1$ for the tokenization NFA $\mathcal{A}$ for $\bar{r}$,
(2) the DFA $\mathcal{A}_2$ for the reverse NFA $\mathcal{A}_{\text{rev}}$, and
(3) a table $test[q, q']$, where q is a state of $\mathcal{A}_1$ and q' is a state of $\mathcal{A}_2$, that corresponds to the test $q \cap q' = \emptyset$ (when q, q' are viewed as NFA powerstates).

Suppose that the subset S (containing states of the forward NFA $\mathcal{A}$) corresponds to a state q of the forward DFA $\mathcal{A}_1$. Similarly, suppose that the subset S' (containing states of the backwards NFA $\mathcal{A}_{\text{rev}}$) corresponds to a state q' of the backwards DFA $\mathcal{A}_2$. The table entry $test[q, q']$

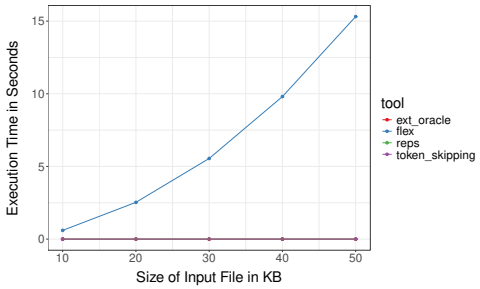
Let M_1 and M_2 be the sizes of the DFAs $\mathcal{A}_1$ ("forward" DFA) and $\mathcal{A}_2$ ("backwards" DFA) respectively. The DFA optimization is activated when both M_1 and M_2 are "small", which we take to mean that they are at most $\max(c_1, c_2 \cdot \sqrt{n}) = O(\sqrt{n})$, where c_1 and c_2 are constants. DFA construction terminates early if the DFAs get larger than the allowed bounds, in which case the algorithm reverts to using NFAs instead. Storing the DFAs $\mathcal{A}_1$ and $\mathcal{A}_2$ requires $O(\sqrt{n})$ memory locations. The table *test* needs $O(M_1 \cdot M_2)$ bits of memory. Since both M_1 and M_2 are $O(\sqrt{n})$, we see that

Complexity. Using these DFA-based data structures, the procedure `Tokenize2` performs $O(1)$ work for every repetition of the main loop. The oracle tape is computed by `ReverseMatch` in $O(n)$ time. It follows that the overall time complexity is $O(n)$. The work space used by the algorithm is dominated by the oracle tape, which needs $O(n)$ memory locations, where each memory location stores a single DFA state.

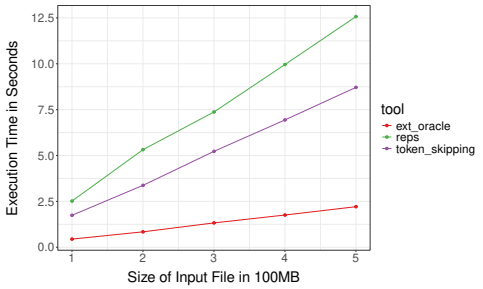
Example: Memory comparison with the algorithm of [Reps 1998]. Let us continue with the earlier example, where we considered input text of length $n = 4\,\text{GB}$ and $M = 100$. We remarked that the algorithm of [Reps 1998] needs 50 GB of memory. Our (optimized) algorithm needs $4 \cdot 10^9$ memory locations, each of which stores a DFA state (1 byte suffices). So, our algorithm needs 4 GB of memory. So, for this particular example, the algorithm of [Reps 1998] needs 12.5× more memory than our algorithm. This gap between the algorithms with respect to memory use becomes greater for larger values of M.

6 Experiments

We have implemented in the Rust programming language our proposed tokenization algorithms: the extension-oracle algorithms (both ExtOracleNoOpt described in Fig. 5 and its optimized variant ExtOracleOpt), and the token-skipping algorithm described in Fig. 4. We compare our algorithms against (1) flex [Vern Paxson 1987], a widely used open-source lexer generator that is based on backtracking DFAs, and (2) our own implementation of the linear-time tokenization algorithm proposed in [Reps 1998]. Through the experiments to be described in this section, we aim to answer the following research questions:



(a) Flex exhibits superlinear behavior.



(b) The other three are mostly linear.

Fig. 7. Evaluation of flex (`flex`) [Vern Paxson 1987], Reps' Algorithm (`reps`) [Reps 1998], our extension-oracle algorithm (`ext_oracle`) in Fig. 5, and our token-skipping algorithm (`token_skipping`) in Fig. 4 using the grammar a and a*b over inputs consisting of many a's.

(1) **RQ1**: Are our proposed algorithms able to efficiently handle certain families of inputs that are known to cause slowdowns in our competitors?
(2) **RQ2**: How do our proposed algorithms perform on practical workloads in various standard data exchange formats?
(3) **RQ3**: How do our proposed algorithms perform on variants of the standard JSON grammar and practical JSON workloads?

In the figures of this section, the label `ext_oracle` refers to the algorithm ExtOracleOpt, which incorporates the DFA-based optimization.

Experimental Setup. All experiments to be described below were conducted on a Linux (Ubuntu) machine equipped with an AMD Ryzen 9 5900 12-core processor clocked at 3.0 GHz and 32 GB of RAM.

6.1 RQ1: Handling Adversarial Workloads

We conducted two microbenchmarks to evaluate our algorithms' robustness in handling inputs that cause performance issues in flex [Vern Paxson 1987] and the algorithm of Reps [1998].

The first microbenchmark uses (i) the tokenization grammar consisting of the rules `a` and `a*b`, and (ii) the input text consists of a large number of `a`'s. The idea is that flex's backtracking algorithm will keep consuming more `a`'s in hopes of finding a `b` to complete a token matching the rule `a*b`. Clearly, such a search is never fruitful, but the algorithm gives up only when it reaches the end of the string. This process is repeated at each index of the input string, resulting in overall quadratic running time. The experimental results for this microbenchmark are shown in Fig. 7. We see in Fig. 7a that flex exhibits superlinear behavior with respect to the input length, taking over 15 seconds to tokenize 50 kilobytes of input. In Fig. 7b, we were able to assess the other linear-time tools on much larger inputs (the unit for the horizontal axis in Fig. 7b is 100 MB). We observe a roughly linear trend in all three tools, with both our proposed algorithms outperforming Reps' algorithm. Comparing Fig. 7a and Fig. 7b, we observe that all the other three tools require less time to process 500 MB of input than flex does to process merely 50 KB of input, which shows that flex's quadratic worst-case complexity can be a major performance issue in certain cases.

The other microbenchmark that we consider is the family of tokenization grammars `a|a{0,k}b`, where k is a parameter. The corresponding input text is the same as before, namely a large number of `a` characters. Fig. 8 shows the experimental results for this microbenchmark. Each of the subplots

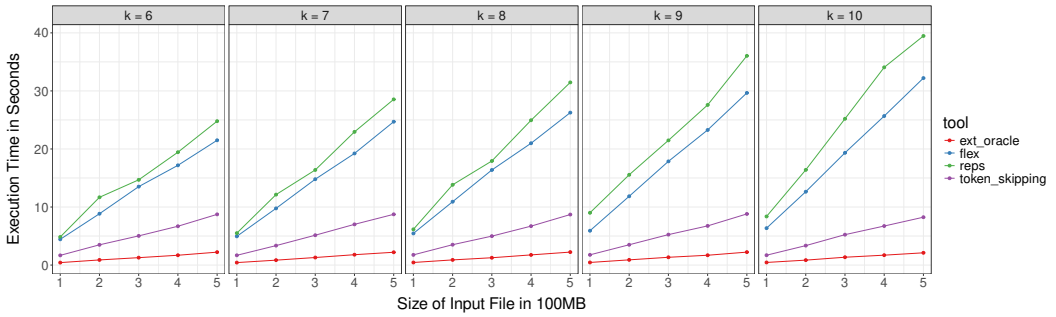


Fig. 8. Evaluation of flex (`flex`) [Vern Paxson 1987], Reps' Algorithm (`reps`) [Reps 1998], our extension-oracle algorithm (`ext_oracle`) in Fig. 5, and our token-skipping algorithm (`token_skipping`) in Fig. 4 using the grammar `a` and `a{0,k}b` over inputs consisting of many `a`'s.

corresponds to a different value of k, as indicated by the title. The idea behind this microbenchmark is similar to the previous one. The difference is that, with a fixed k, the amount of backtracking flex [Vern Paxson 1987] must perform *at each step* is bounded by the constant k, whereas, previously in the case of `a|a*b`, the amount of backtracking flex performs *at each step* is linear in the size of the input. Thus, in this microbenchmark, flex has linear time complexity. Nevertheless, it still performs poorly compared to our token-skipping and extension-oracle algorithms despite its linear growth in running time. More interestingly, observe that Reps' algorithm performs worse than in the previous microbenchmark. This is because Reps' algorithm requires that we fill in an entry in the memoization table for every DFA state and input index pair that has been explored and known to lead to a failure. This can become prohibitive in practice when we have to fill in many entries in the table. For the previous microbenchmark (`a|a*b`), the DFA has 5 states. For $k = 10$ here (`a|a{0,10}b`), the DFA has $13 = 2.6 \times 5$ states. Indeed, we do observe that the running time of Rep's algorithm on `a|a{0,10}b` is about 3× of that on `a|a*b` on all input sizes. On the other hand, the two algorithms that we propose do not suffer from these issues and are much more efficient in these cases.

6.2 RQ2: Performance for Standard Data Exchange Formats

As mentioned in the introduction, tokenization could serve as an efficient way to extract information from text-based data formats without using context-free parsing, which is more computationally expensive. We tested the performance of our proposed algorithms in tokenizing documents of popular data exchange formats such as JSON, CSV, and TSV. The standard JSON grammar is shown below. The grammar rules on the first line correspond to the valid keywords and delimiters in JSON. The rule on the second line is meant to capture numeric values in JSON, and the rule on the third line captures valid strings in JSON.

```
null    false    true    {    }    [    ]    ,    :
-?(0|[1-9]\d*)(\.\d+)?([eE][+-]?\d+)?
\"([^"\\]|\\["\\/bnfrt]|\\u[a-fA-F0-9]{4})*\"
```

The tokenization grammar for CSV was previously introduced in Example 3. The grammar for TSV is very similar, except that the field delimiter is a tab character `\t` in lieu of a comma and that there are no quoted fields in TSV. Additionally, we also consider two domain-specific data formats: FASTA and DNS zone files. The FASTA format [Lipman and Pearson 1985] is widely used

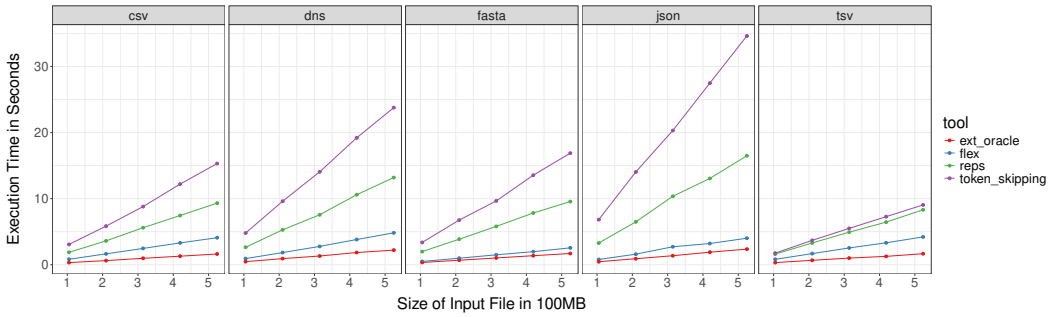


Fig. 9. Evaluation of flex (`flex`) [Vern Paxson 1987], Reps' Algorithm (`reps`) [Reps 1998], our extension-oracle algorithm (`ext_oracle`) in Fig. 5, and our token-skipping algorithm (`token_skipping`) in Fig. 4 on different data exchange formats.

Table 1. Throughput of flex (`flex`) [Vern Paxson 1987], Reps' Algorithm (`reps`) [Reps 1998], our extension-oracle algorithm (`ext_oracle`) in Fig. 5, and our token-skipping algorithm (`token_skipping`) in Fig. 4 on different data exchange formats. The measurement unit is megabytes per second (MBps).

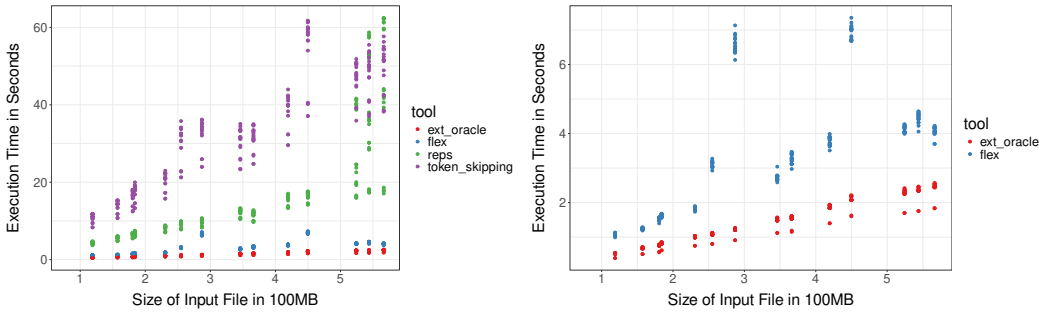
	CSV	DNS	FASTA	JSON	TSV
Extension-oracle (Fig. 5 with DFA opt.)	323.6	237.8	309.1	222.3	314.5
flex [Vern Paxson 1987]	128.3	108.8	205.4	130.8	124.4
Reps [Reps 1998]	56.3	39.8	54.9	31.8	63.0
Token-skipping (Fig. 4)	34.2	22.1	31.1	15.1	57.8

in bioinformatics to encode nucleotide or amino acid sequences. DNS zone files, as introduced by RFC 1035 [Mockapetris 1987] and RFC 2308 [Andrews 1998], are text-based formats that describe DNS zones. We obtained the JSON arXiv dataset [arXiv.org 2024] and the CSV/TSV Amazon Book Reviews dataset [Amazon 2024] from Kaggle [Kaggle 2010]. Further, we retrieved files that encode nucleotide sequences in the FASTA format from [of Medicine 2021]. The DNS zone files were published in [Koekkoek and Lemire 2024]. For each of these datasets, we took five prefixes approximately of sizes 100 MB, 200 MB, 300 MB, 400 MB, and 500 MB, respectively. These prefixes served as the input text in our experiments.

Fig. 9 shows the evaluation results using the standard grammars. Each of the five subplots corresponds to a data exchange format, as indicated in the subplot title. The vertical axis shows the running time of each tool in seconds, and the horizontal axis shows the size of the input files in hundreds of megabytes. All four tools exhibit a roughly linear running-time growth on all five data exchange formats. Across all five data exchange formats, our extension-oracle algorithm is consistently the fastest. Due to the overhead of NFA simulation, our token-skipping algorithm tends to be slower than the other three DFA-based algorithms on realistic benchmarks. Moreover, since all four tools exhibit mostly linear running-time growths, we compute the throughput of each tool on each data format at input size $\approx$ 500 MB and present the results in Table 1. The measurement unit is megabytes per second (MBps).

6.3 RQ3: Using JSON Variants

We previously mentioned that developers might want to modify existing data exchange formats to suit their own preferences and needs. To test the efficiency of our algorithms in this case, we



(a) All four tools on JSON Variants (b) Extension-oracle and flex on JSON Variants

Fig. 10. Evaluation of flex (`flex`) [Vern Paxson 1987], Reps' Algorithm (`reps`) [Reps 1998], our extension-oracle algorithm (`ext_oracle`) in Fig. 5, and our token-skipping algorithm (`token_skipping`) in Fig. 4 using twenty variants of JSON grammar and realistic JSON input texts.

crafted variants of the standard JSON grammar. These variants are combinations of modifications of the standard grammar motivated by potential use cases in practice. For example, the standard JSON grammar has one rule that captures all valid numeric values. However, one may wish to refine the rule to distinguish between different types of numeric values such as integers, floats, and floats with scientific notation, respectively. This could be useful if the user would like to extract a specific type of value from the file without full parsing. Thus, we consider a grammar variant where we replace the original number-matching rule in the standard grammar with the following three rules, where the first two capture integers and floats, respectively, and the third captures specifically floats with scientific notation.

```
-?(0|[1-9]\d*)      -?(0|[1-9]\d*)(\.\d+)      -?(0|[1-9]\d*)(\.\d+)([eE][+-]?\d+)
```

Following a similar methodology, we created 20 such grammar variants, all of which were used in the performance evaluation. For the input text, we have collected 14 large JSON files from online sources such as Kaggle [Kaggle 2010] and data.gov [data.gov 2009]. These files are of sizes ranging from 119 to 566 megabytes. The files also feature a diverse set of realistic application domains, including health (e.g., Alzheimer's disease statistics), entertainment (e.g., video game reviews), and technology (e.g., bitcoin transactions).

Fig. 10 shows the evaluation results using 20 JSON variants. The left subfigure (Fig. 10a) plots all four tools, and the right subfigure (Fig. 10b) plots only our extension-oracle algorithm and flex [Vern Paxson 1987] to facilitate visual comparisons between these two best-performing tools. Each data point in the graph corresponds to one instance of an experiment where a different tool, grammar variant, and input text are used. The x-axis measures the size of the input file in hundreds of megabytes, and the y-axis measures the execution time in seconds. Overall, we observe a somewhat similar hierarchy in performance as we did in Fig. 9, where our extension-oracle algorithm is the best-performing across the board. Although flex [Vern Paxson 1987] does outperform the other two algorithms, without the linear-time guarantee, we do observe conspicuous spikes in flex's running time at approximately $x = 2.9$ and $x = 4.5$. On the other hand, while Reps' algorithm [Reps 1998] exhibits a stable performance for smaller inputs, it struggles to handle larger inputs due to the prohibitive amounts of memory it requires. In fact, when the machine runs out of RAM, the operating system must start *swapping*, taking pages from the RAM and putting them on the disks. Data access then becomes much slower, leading to a degradation in time efficiency.

Main findings. Our experimental evaluation has empirically confirmed the linear-time guarantees of our algorithms described in Fig. 4 and Fig. 5. Moreover, the experiments have shown that, for realistic data, our proposed extension-oracle algorithm in Fig. 6 consistently outperforms the fast lexing tool flex [Vern Paxson 1987] by 2× to 3× and outperforms the memoization-based algorithm proposed by Reps [Reps 1998] by about 5×. Further, while flex's backtracking-based algorithm may exhibit quadratic behavior and Reps' algorithm may experience a significant slowdown due to its heavy space usage, both our proposed algorithms are impervious to such issues. Overall, the experimental results show that practical workloads favor DFA-based implementation.

6.4 Discussion on the Performance of the Proposed Algorithms

We remind the reader that in the figures of this section the label `token_skipping` refers to the TokenSkip algorithm and the label `ext_oracle` refers to the ExtOracleOpt algorithm, which includes the DFA-based optimization.

The token-skipping algorithm (TokenSkip) is slower than ExtOracleOpt because the reverse (i.e., right-to-left) pass of TokenSkip performs an NFA simulation. Each active state carries a position in the string, which is an additional overhead compared to a pure Boolean NFA simulation.

The implementation of the extension-oracle algorithm ExtOracleOpt integrates the performance optimization of using DFA simulation when the DFAs are "small" (see §5.3). For practical workloads, the DFA size does not explode and the DFA optimization gets activated. This is also the case for the workloads of the experimental evaluation. DFA simulation is generally faster than NFA simulation, which explains why the extension-oracle algorithm has higher throughput than the token-skipping algorithm.

7 Related Work

Tokenization and lexing algorithms. Tokenization is also referred to as lexing, especially in the context of lexical analysis for compilers. Standard textbook algorithms for lexing are automata-based and are well-documented in classic references such as [Aho et al. 2007]. In theory, this standard algorithm may exhibit quadratic-time behaviors due to backtracking. The lexer-generator tool flex [Vern Paxson 1987] implements the technique described in [Aho et al. 2007], and as we have seen empirically, flex suffers from quadratic running time in some cases. On the other hand, Reps [1998] proposed a linear-time lexing algorithm using memoization, which avoids the repeated exploration of paths that the algorithm has already seen that they eventually fail. Since the memoization table must keep track of both the position in the string and the state in the automaton, Reps' approach requires $O(M \cdot (n+1))$ additional bits of memory, where M is the number of DFA states and n is the length of the input string. Most notable lexer generators used in practice, including flex [Vern Paxson 1987], Ragel [Colm Networks 2021], Re/flex [Robert van Engelen 2016], are DFA-based despite the fact that DFAs may be of exponential size. We have observed that the regular expressions used in lexing/tokenization are typically deterministic, which is why having a substantial blowup in the DFA size is rare in practice.

Recent works in the area lexing and tokenization have investigated parallelization and formal verification. Plex is an automaton-based parallel lexing scheme proposed in [Li et al. 2021], where the additional prescanning phase helps identify context information that in turn enables parallelization. In verified lexing, works such as Verbatim [Egolf et al. 2021], Verbatim++ [Egolf et al. 2022], and Coqlex [Ouedraogo et al. 2023] have emerged in recent years. Motivated by the fact that the front ends of a safety-critical system, which includes lexing, are often the target of security attacks, these works produce verified lexers that are mathematically guaranteed to conform to the standard maximal-munch semantics. These works are based on Brzozowski [1964] derivatives.

Lookahead. Performing a reverse pass over the input string has been considered in the context of temporal monitoring using alternating finite automata [Finkbeiner and Sipma 2004]. This idea of processing the input string in reverse is already present in the original work on the notion of alternation [Chandra et al. 1981] (see page 128 and the definition of the function G_i "outside in"). There is also recent work on matching regular expressions with lookaround assertions (i.e., lookbehind and lookahead assertions) that uses backwards passes over the input stream to deal with lookahead, e.g., in [Mamouras and Chattopadhyay 2024] and [Barrière and Pit-Claudel 2024].

Information extraction using regular languages. We have mentioned that regular tokenization can, in some cases, serve as an alternative to full context-free parsing when it comes to information extraction, especially when the desired information is context-independent and the data is too large to be parsed easily. Indeed, this methodology has been pursued by works such as [Grathwohl et al. 2016; Nourian et al. 2022]. In particular, Grathwohl et al. [2016] proposed a framework named Kleenex for extracting information from streams based on regular expressions. Related to this line of work is the area of regular expression parsing, which is typically done under either the greedy/PCRE2 disambiguation policy [The PCRE2 Developers 2024] or the POSIX disambiguation policy [POSIX 2008]. One of the most important works in the area of greedy regular expression parsing is [Frisch and Cardelli 2004], where the authors proposed that regular expressions could give rise to structured representations of input text values instead of a flat sequence. Following [Frisch and Cardelli 2004], Grathwohl et al. [2014] studied the problem of greedy parsing in a streaming setting.

On the other hand, POSIX regular expression parsing is a difficult problem despite the fact that the POSIX disambiguation policy itself is quite elegant and intuitive - arguably more so than the greedy/PCRE policy. An important work in the area of POSIX regular expression parsing is [Sulzmann and Lu 2014], where the authors described a Brzozowski-derivative-based algorithm that generates POSIX parse trees. More works such as [Ausaf et al. 2016; Tan and Urban 2023; Urban 2023] follow, which formally verify derivative-based POSIX parsing algorithms in the proof assistant Isabelle/HOL.

8 Conclusion

Motivated by the extensive practical applications of tokenization in various domains, we have studied the uniform tokenization problem and proposed two novel and efficient algorithms. These two NFA-based algorithms have guaranteed linear time complexity, which allows them to circumvent two potential performance issues commonly found in existing tools: the worst-case quadratic complexity due to backtracking and the worst-case exponential complexity due to the use of DFAs. In particular, our extension-oracle algorithm in Fig. 5 has time complexity $O(mn)$ and space complexity $O(mn)$, and our token-skipping algorithm in Fig. 4 has time complexity $O(mn)$ and space complexity $O(m + n)$.

We have empirically compared our proposed algorithms against the standard tool flex [Vern Paxson 1987] and the memoization-based algorithm in [Reps 1998]. The experiments confirmed the linear growth of our algorithms. In particular, our extension-oracle algorithm proves to be 2× to 3× faster than all other algorithms. Thus, not only do we guarantee linear-time complexity, but we also improve upon previous memory bounds and demonstrate better time efficiency on practical workloads.

Data-Availability Statement

The data and software used for the experiments of the paper can be found here.

Acknowledgments

We would like to thank the anonymous reviewers for their constructive comments. This research was supported in part by the US National Science Foundation award CCF 2340479.

References

Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. 2007. *Compilers Principles, Techniques & Tools*. Pearson Education.

Amazon. 2024. Amazon Book Reviews Dataset. https://www.kaggle.com/datasets/mohamedbakhet/amazon-books-reviews

M. Andrews. 1998. RFC2308: Negative Caching of DNS Queries (DNS NCACHE). https://datatracker.ietf.org/doc/html/rfc2308

arXiv.org. 2024. arXiv Dataset. https://doi.org/10.34740/KAGGLE/DSV/7548853

Fahad Ausaf, Roy Dyckhoff, and Christian Urban. 2016. POSIX Lexing with Derivatives of Regular Expressions (Proof Pearl). In *Interactive Theorem Proving (ITP 2016) (Lecture Notes in Computer Science, Vol. 9807)*, Jasmin Christian Blanchette and Stephan Merz (Eds.). Springer, Cham, 69–86. https://doi.org/10.1007/978-3-319-43144-4_5

Alessandro Barenghi, Stefano Crespi Reghizzi, Dino Mandrioli, Federica Panella, and Matteo Pradella. 2015. Parallel Parsing Made Practical. *Science of Computer Programming* 112 (2015), 195–226. https://doi.org/10.1016/j.scico.2015.09.002

Aurèle Barrière and Clément Pit-Claudel. 2024. Linear Matching of JavaScript Regular Expressions. *Proceedings of the ACM on Programming Languages* 8, PLDI, Article 201 (June 2024), 25 pages. https://doi.org/10.1145/3656431

Janusz A. Brzozowski. 1964. Derivatives of Regular Expressions. *J. ACM* 11, 4 (1964), 481–494. https://doi.org/10.1145/321239.321249

Ashok K. Chandra, Dexter C. Kozen, and Larry J. Stockmeyer. 1981. Alternation. *J. ACM* 28, 1 (1981), 114–133. https://doi.org/10.1145/322234.322243

Agnishom Chattopadhyay, Angela W. Li, and Konstantinos Mamouras. 2025. Verified and Efficient Matching of Regular Expressions with Lookaround. In *Proceedings of the 14th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP '25)*. ACM, New York, NY, USA, 198–213. https://doi.org/10.1145/3703595.3705884

Colm Networks. 2021. Ragel State Machine Compiler. https://www.colm.net/open-source/ragel/. [Online; accessed October 15, 2024].

Russ Cox. 2010. Regular Expression Matching in the Wild. https://swtch.com/~rsc/regexp/regexp3.html. [Online; accessed October 15, 2024].

data.gov. 2009. U.S. Government's Open Data. https://data.gov/

Derek Egolf, Sam Lasser, and Kathleen Fisher. 2021. Verbatim: A Verified Lexer Generator. In *2021 IEEE Security and Privacy Workshops (SPW)*. IEEE, USA, 92–100. https://doi.org/10.1109/SPW53761.2021.00022

Derek Egolf, Sam Lasser, and Kathleen Fisher. 2022. Verbatim++: Verified, Optimized, and Semantically Rich Lexing with Derivatives. In *Proceedings of the 11th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2022)*. ACM, New York, NY, USA, 27–39. https://doi.org/10.1145/3497775.3503694

Bernd Finkbeiner and Henny Sipma. 2004. Checking Finite Traces Using Alternating Automata. *Formal Methods in System Design* 24, 2 (2004), 101–127. https://doi.org/10.1023/B:FORM.0000017718.28096.48

Alain Frisch and Luca Cardelli. 2004. Greedy Regular Expression Matching. In *Automata, Languages and Programming (ICALP 2004) (Lecture Notes in Computer Science, Vol. 3142)*. Springer, Berlin, Heidelberg, 618–629. https://doi.org/10.1007/978-3-540-27836-8_53

Bjørn Bugge Grathwohl, Fritz Henglein, Ulrik Terp Rasmussen, Kristoffer Aalund Søholm, and Sebastian Paaske Tørholm. 2016. Kleenex: Compiling Nondeterministic Transducers to Deterministic Streaming Transducers. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '16)*. ACM, New York, NY, USA, 284–297. https://doi.org/10.1145/2837614.2837647

Niels Bjørn Bugge Grathwohl, Fritz Henglein, and Ulrik Terp Rasmussen. 2014. Optimally Streaming Greedy Regular Expression Parsing. In *Theoretical Aspects of Computing – ICTAC 2014 (Lecture Notes in Computer Science, Vol. 8687)*, Gabriel Ciobanu and Dominique Méry (Eds.). Springer, Cham, 224–240. https://doi.org/10.1007/978-3-319-10882-7_14

JSON. 2001. JSON. https://www.json.org/json-en.html [Online; accessed October 15, 2024].

JSON5. 2012. JSON5: JSON for humans. https://json5.org/ [Online; accessed October 15, 2024].

Kaggle. 2010. Kaggle. https://www.kaggle.com

Jeroen Koekkoek and Daniel Lemire. 2024. Parsing Millions of DNS Records per Second. *Software: Practice and Experience* 55, 4 (2024), 778–788. https://doi.org/10.1002/spe.3396

Lingkun Kong, Qixuan Yu, Agnishom Chattopadhyay, Alexis Le Glaunec, Yi Huang, Konstantinos Mamouras, and Kaiyuan Yang. 2022. Software-Hardware Codesign for Efficient In-Memory Regular Pattern Matching. In *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI 2022)*. ACM, New York, NY, USA, 733–748. https://doi.org/10.1145/3519939.3523456

Alexis Le Glaunec, Lingkun Kong, and Konstantinos Mamouras. 2023. Regular Expression Matching using Bit Vector Automata. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1, Article 92 (2023), 30 pages. https://doi.org/10.1145/3586044

Le Li, Shigeyuki Sato, Qiheng Liu, and Kenjiro Taura. 2021. Plex: Scaling Parallel Lexing with Backtrack-Free Prescanning. In *2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, USA, 693–702. https://doi.org/10.1109/IPDPS49936.2021.00079

David J. Lipman and William R. Pearson. 1985. Rapid and Sensitive Protein Similarity Searches. *Science* 227, 4693 (1985), 1435–1441. https://doi.org/10.1126/science.2983426

Konstantinos Mamouras and Agnishom Chattopadhyay. 2024. Efficient Matching of Regular Expressions with Lookaround Assertions. *Proceedings of the ACM on Programming Languages* 8, POPL, Article 92 (2024), 31 pages. https://doi.org/10.1145/3632934

Konstantinos Mamouras, Alexis Le Glaunec, Wu Angela Li, and Agnishom Chattopadhyay. 2024. Static Analysis for Checking the Disambiguation Robustness of Regular Expressions. *Proceedings of the ACM on Programming Languages* 8, PLDI, Article 231 (2024), 25 pages. https://doi.org/10.1145/3656461

Takayuki Miyazaki and Yasuhiko Minamide. 2019. Derivatives of Regular Expressions with Lookahead. *Journal of Information Processing* 27 (2019), 422–430. https://doi.org/10.2197/ipsjjip.27.422

Paul Mockapetris. 1987. RFC 1035: Domain Names - Implementation and Specification. https://datatracker.ietf.org/doc/html/rfc1035

Marziyeh Nourian, Tri Nguyen, Andrew A. Chien, and Michela Becchi. 2022. Data Transformation Acceleration using Deterministic Finite-State Transducers. In *2022 IEEE International Conference on Big Data (Big Data)*. IEEE, USA, 141–150. https://doi.org/10.1109/BigData55660.2022.10020756

National Library of Medicine. 2021. FASTA Format for Nucleotide Sequences. (2021). https://www.ncbi.nlm.nih.gov/genbank/fastaformat/

Wendlasida Ouedraogo, Gabriel Scherer, and Lutz Straßburger. 2023. Coqlex: Generating Formally Verified Lexers. *The Art, Science, and Engineering of Programming* 8, 1, Article 3 (2023), 30 pages. https://doi.org/10.22152/programming-journal.org/2024/8/3

POSIX. 2008. IEEE Standard for Information Technology - Portable Operating System Interface (POSIX(R)). *IEEE Std 1003.1-2008 (Revision of IEEE Std 1003.1-2004)* (2008), 1–3874. https://doi.org/10.1109/IEEESTD.2008.4694976

Thomas Reps. 1998. "Maximal-Munch" Tokenization in Linear Time. *ACM Transactions on Programming Languages and Systems* 20, 2 (1998), 259–273. https://doi.org/10.1145/276393.276394

Robert van Engelen. 2016. RE/flex: A high-performance C++ regex library and a lexical analyzer generator. https://www.genivia.com/doc/reflex/html/. [Online; accessed October 15, 2024].

Yakov Shafranovich. 2005. RFC4180: Common Format and MIME Type for Comma-Separated Values (CSV) Files. https://www.ietf.org/rfc/rfc4180.txt

Joshua B. Smith. 2007. *Ocamllex and Ocamlyacc.* Apress, Berkeley, CA, 193–211. https://doi.org/10.1007/978-1-4302-0244-8_16

Martin Sulzmann and Kenny Zhuo Ming Lu. 2014. POSIX Regular Expression Parsing with Derivatives. In *Functional and Logic Programming (FLOPS 2014) (Lecture Notes in Computer Science, Vol. 8475)*, Michael Codish and Eijiro Sumii (Eds.). Springer, Cham, 203–220. https://doi.org/10.1007/978-3-319-07151-0_13

Chengsong Tan and Christian Urban. 2023. POSIX Lexing with Bitcoded Derivatives. In *14th International Conference on Interactive Theorem Proving (ITP 2023) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 268)*, Adam Naumowicz and René Thiemann (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 27:1–27:18. https://doi.org/10.4230/LIPIcs.ITP.2023.27

The PCRE2 Developers. 2024. Perl-compatible Regular Expressions (revised API: PCRE2). https://pcre2project.github.io/pcre2/doc/html/index.html. [Online; accessed October 15, 2024].

Ken Thompson. 1968. Programming Techniques: Regular Expression Search Algorithm. *Commun. ACM* 11, 6 (1968), 419–422. https://doi.org/10.1145/363347.363387

Christian Urban. 2023. POSIX Lexing with Derivatives of Regular Expressions. *Journal of Automated Reasoning* 67, 3, Article 24 (2023), 24 pages. https://doi.org/10.1007/s10817-023-09667-1

Vern Paxson. 1987. Flex: The Fast Lexical Analyzer. https://github.com/westes/flex. [Online; accessed October 15, 2024].

Ziyuan Wen, Lingkun Kong, Alexis Le Glaunec, Konstantinos Mamouras, and Kaiyuan Yang. 2024. BVAP: Energy and Memory Efficient Automata Processing for Regular Expressions. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. ACM, New York, NY, USA, 151–166. https://doi.org/10.1145/3620665.3640412

XML. 1996. Extensible Markup Language (XML). https://www.w3.org/TR/xml/ [Online; accessed October 15, 2024].

Received 2024-10-16; accepted 2025-02-18